

Introduction to Algorithm

기초 알고리즘

다양한 정렬 알고리즘



정렬 (Sorting)

Q. 정렬이 필요한 이유?

A. 오름차순 및 내림차순으로 정렬되어 있다면 특정 원소를 좀 더 효율적으로 찾을 수 있음

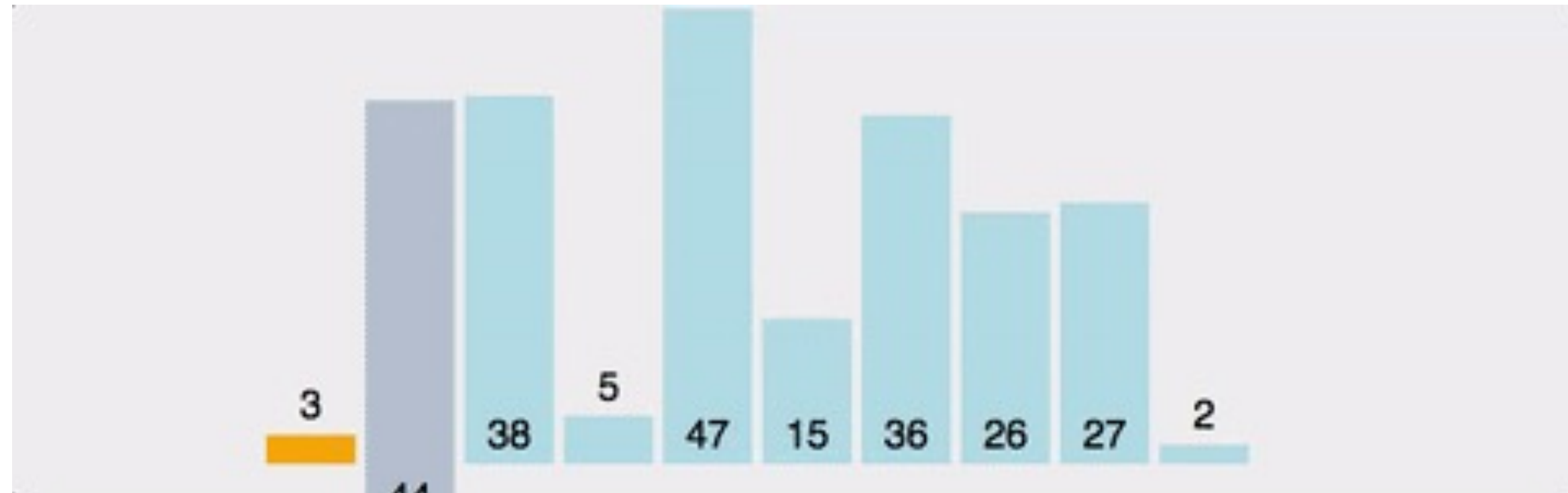
이름	최선	평균	최악	평균 공간 복잡도
삽입 정렬 (Insertion Sort)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
버블 정렬 (Bubble Sort)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
선택 정렬 (Selection Sort)	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
합병 정렬(Merge Sort)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
퀵 정렬(Quick Sort)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
힙 정렬(Heap Sort)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

특정 정렬이 빠르다고 항상 좋은 것은 아님
데이터의 특성, 크기에 따라 적절한 정렬 방법을 사용해야 함



정렬 (Sorting)

삽입 정렬(Insertion Sort)



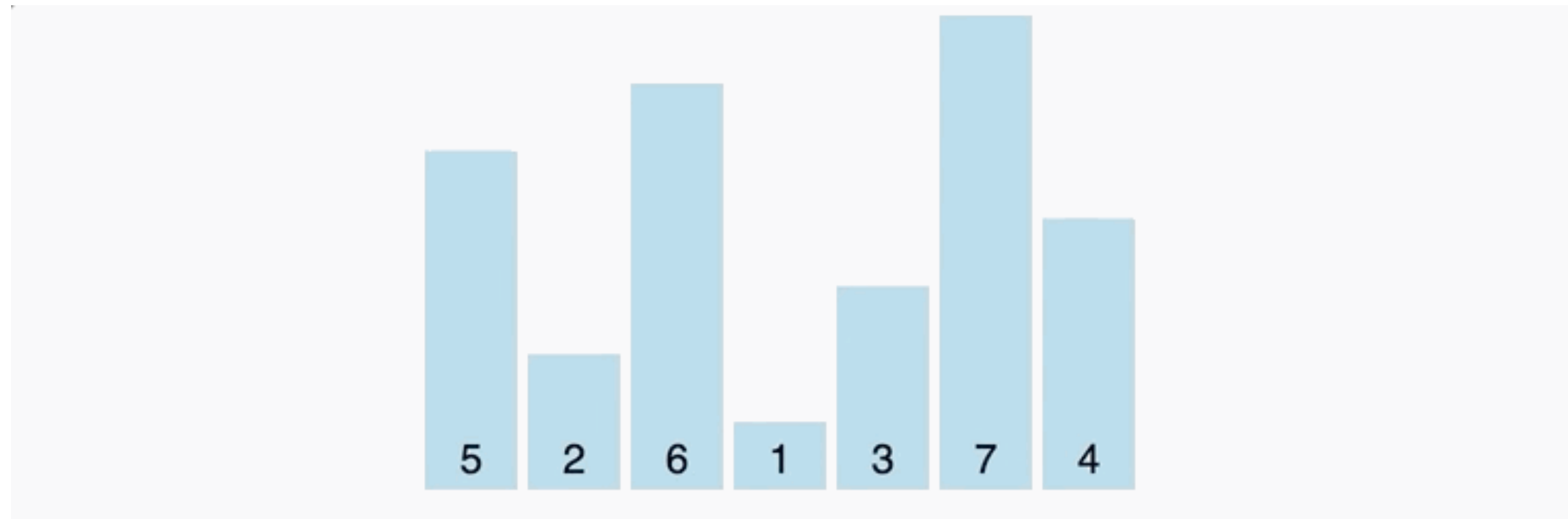
- 배열의 모든 원소를 앞에서부터 차례대로 비교 → 자신의 위치를 찾아 삽입
- 구현이 간단하지만, 배열이 길어질수록 효율이 떨어짐
- Python은 팀 정렬(Tim Sort) : 합병 정렬 + 삽입 정렬



정렬 (Sorting)

버블 정렬 (Bubble Sort)

- 배열의 인접한 두 수를 선택하여, 정렬되지 않았다면 두 수를 바꿈(swap)
- 이를 배열의 처음부터 끝까지 반복하고, 배열에 아무 변화가 없을 때까지 함
- 구현이 간단하지만, 교환횟수가 많고 느림 → 실무에서는 사용되지 않음!

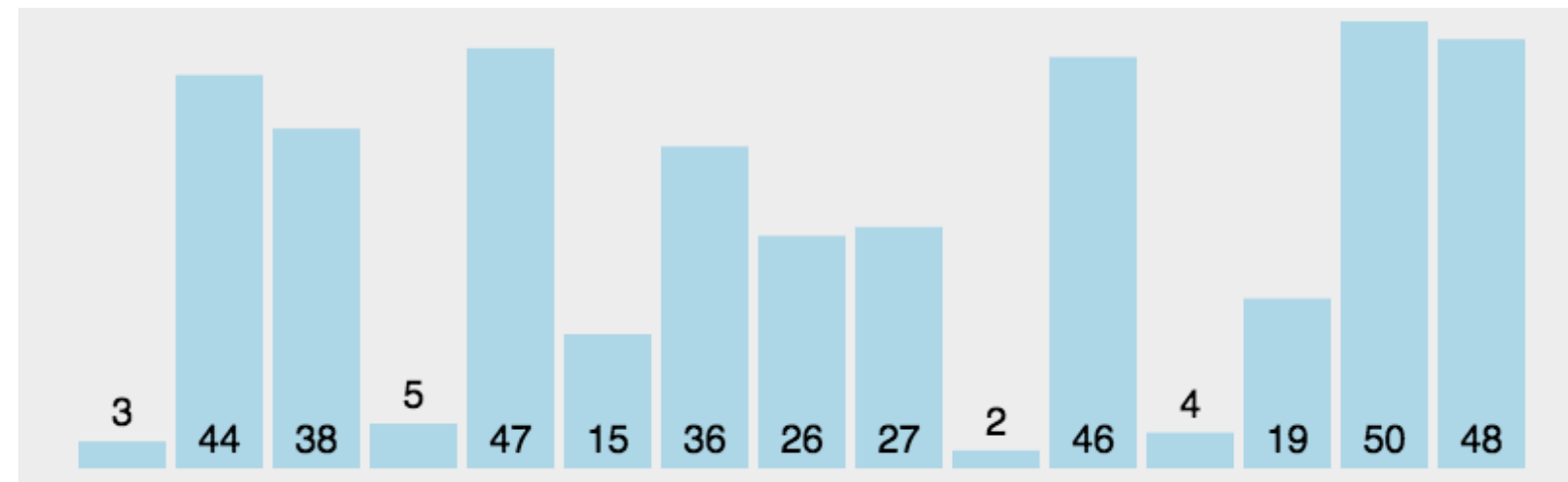




정렬 (Sorting)

선택 정렬 (Selection Sort)

- 매 단계마다 전체에서 가장 작은 값(또는 큰 값)을 선택해서 앞자리에 둬
- 구현이 간단하지만, 대규모 데이터에 부적합 (이미 정렬되어 있어도 **항상 끝까지 탐색**)
- 삽입 정렬이나 버블 정렬과 비교했을 때 → 비교는 많지만, 교환은 적음





정렬 (Sorting)

힙 정렬(Heap Sort)

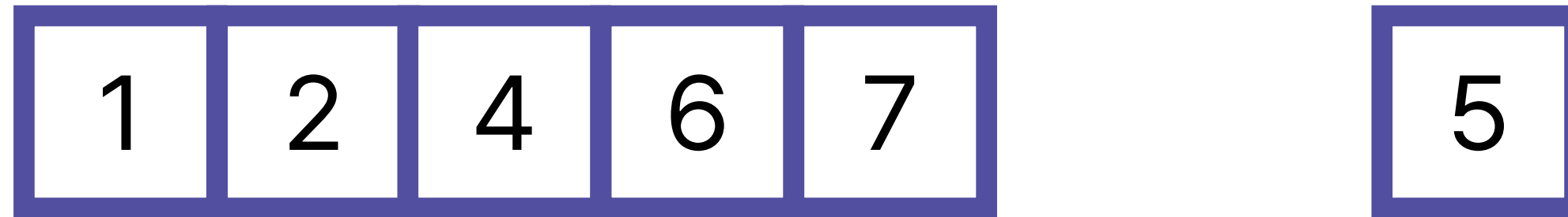
- 최대 힙 구성
 - 최대 힙이란 부모 노드가 자식 노드보다 큰 트리를 의미
- 현재 힙에서 가장 큰 값(루트의 값)을 마지막 요소와 바꾼 후, 힙의 사이즈를 하나 줄이고 다시 최대 힙 구성
- 힙의 사이즈가 1이 될 때까지 위 과정을 반복
- 최악의 경우에도 $O(N \log N)$ 보장하지만, 구현이 어려움

6 5 3 1 8 7 2 4

삽입 정렬



삽입 정렬 (Insertion Sort)



이미 정렬된 배열이 있다고 가정할 때,
이 배열에 추가적인 데이터 하나를 넣고 싶다면?



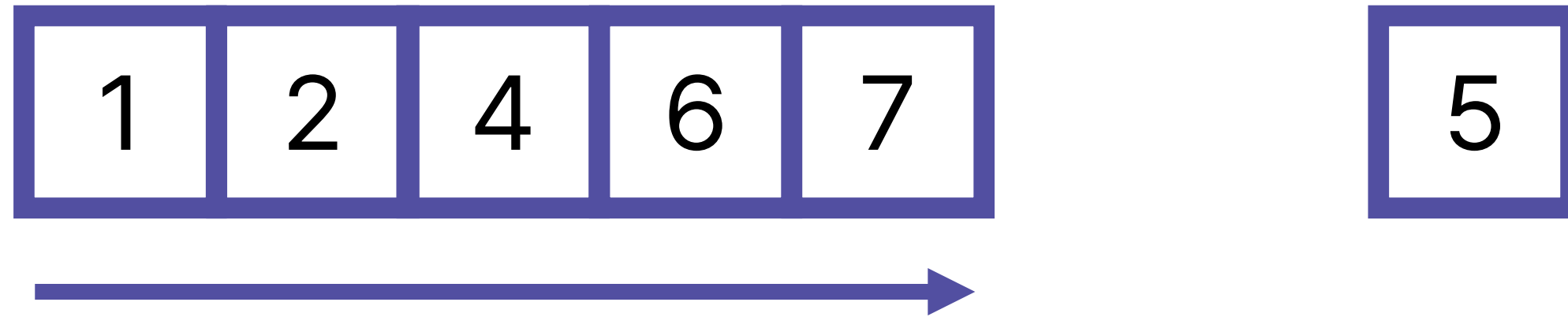
삽입 정렬 (Insertion Sort)



데이터를 넣은 이후에도 정렬된 상태가 유지되어야 하니,
4와 6 사이에 5를 넣어야 함



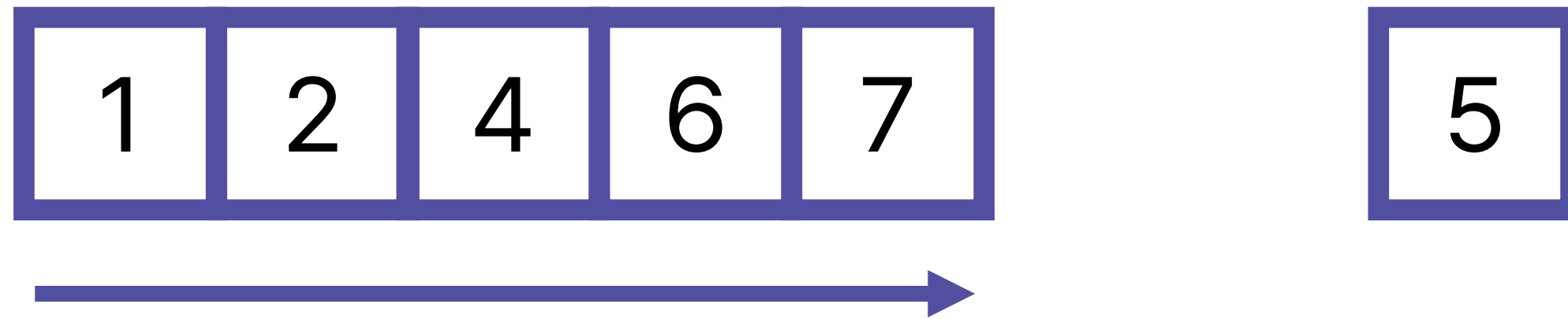
삽입 정렬 (Insertion Sort)



코드로 작성해야 한다면, 배열의 앞부분부터 확인해가며
새 데이터가 들어가야 할 공간을 찾아야 할 것



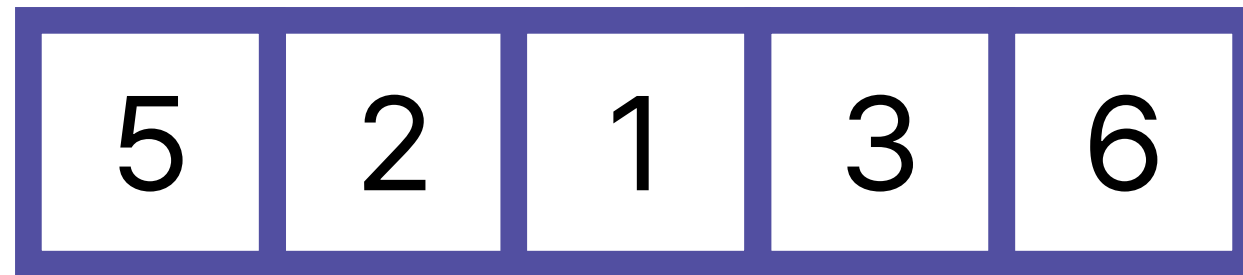
삽입 정렬 (Insertion Sort)



삽입 정렬은 이런 방법을 활용한 정렬 알고리즘



삽입 정렬 (Insertion Sort)



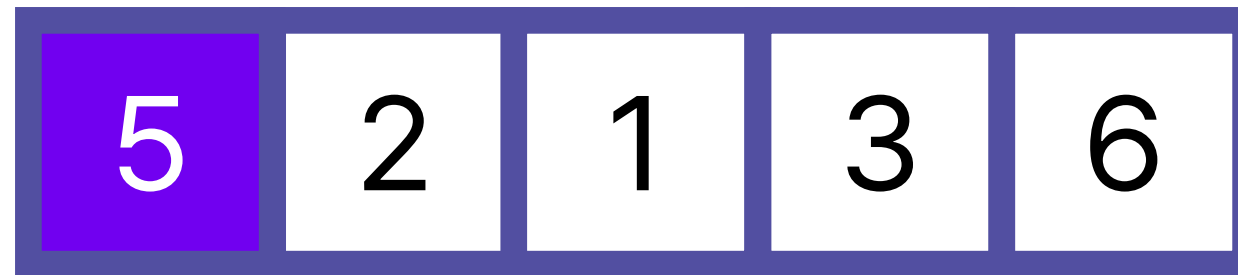
삽입 정렬은 값을 1개씩 늘려가며 정렬을 진행

1개의 원소를 정렬하면, 수를 이동할 필요 없이

그 자체로 정렬된 상태라고 볼 수 있음



삽입 정렬 (Insertion Sort)



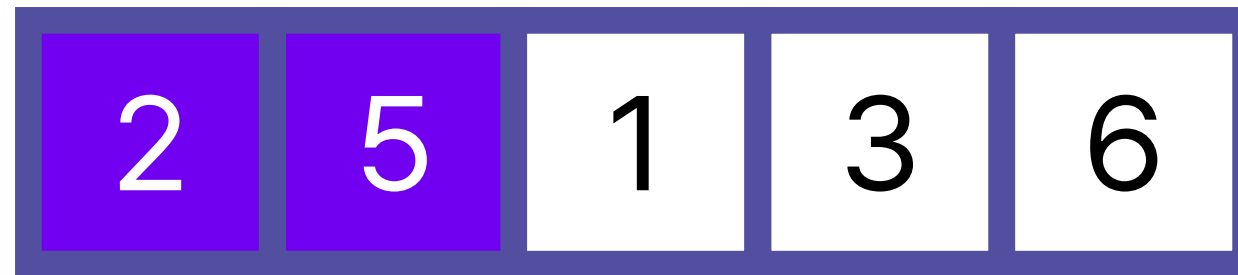
이후, 두 번째 숫자부터 앞에서 설명했던

삽입 방식을 진행

두 번째 숫자인 2는 어디에 넣어야 할까?



삽입 정렬 (Insertion Sort)

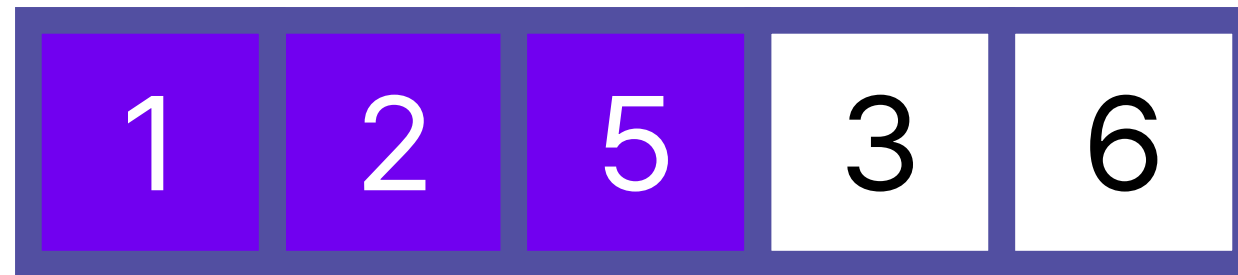


5보다 작으므로, 5의 앞에 2를 삽입해야 함

세 번째 숫자인 1을 삽입하려면?



삽입 정렬 (Insertion Sort)



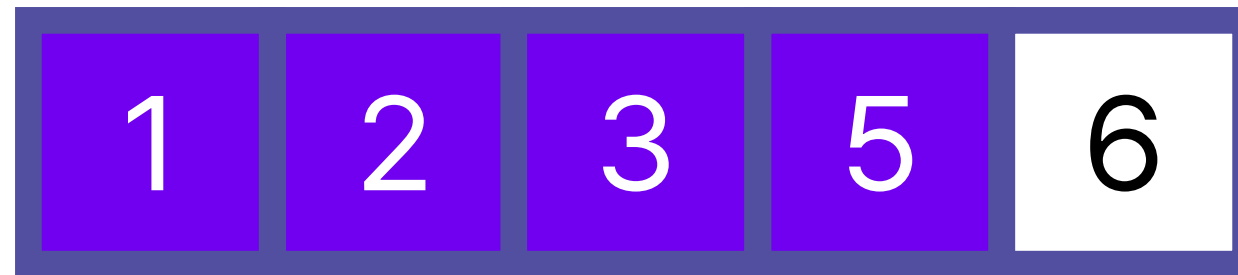
자연스럽게, 맨 앞으로 가야 함

이런 식으로, 남은 수도 들어갈 수 있는 곳을 찾아

그 수를 삽입하면 삽입 정렬이 끝나게 됨



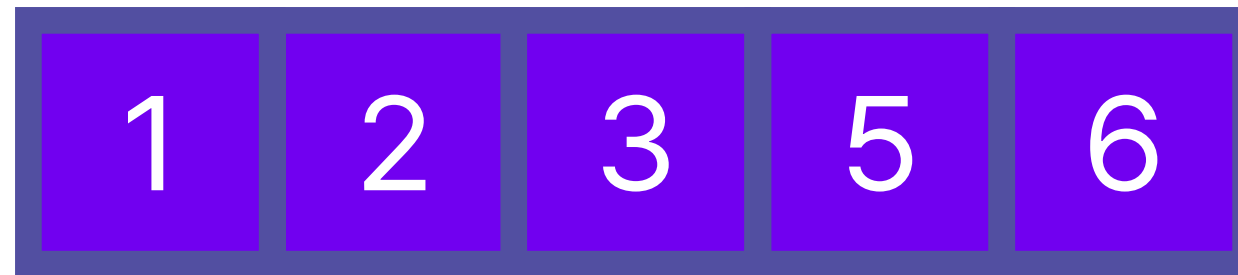
삽입 정렬 (Insertion Sort)



이런 식으로, 남은 수도 들어갈 수 있는 곳을 찾아
그 수를 삽입하면 삽입 정렬이 끝나게 됨



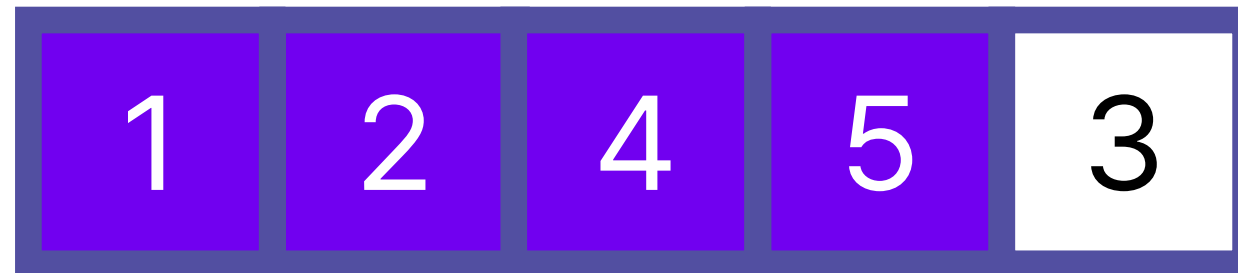
삽입 정렬 (Insertion Sort)



이런 식으로, 남은 수도 들어갈 수 있는 곳을 찾아
그 수를 삽입하면 삽입 정렬이 끝나게 됨



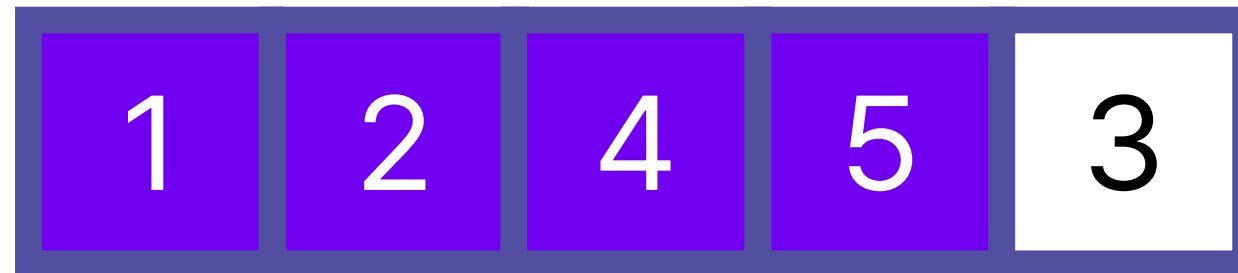
삽입 정렬 (Insertion Sort)



삽입 정렬을 코드로 구현하게 되면,
크게 두 가지 부분으로 나뉘게 됨



삽입 정렬 (Insertion Sort)

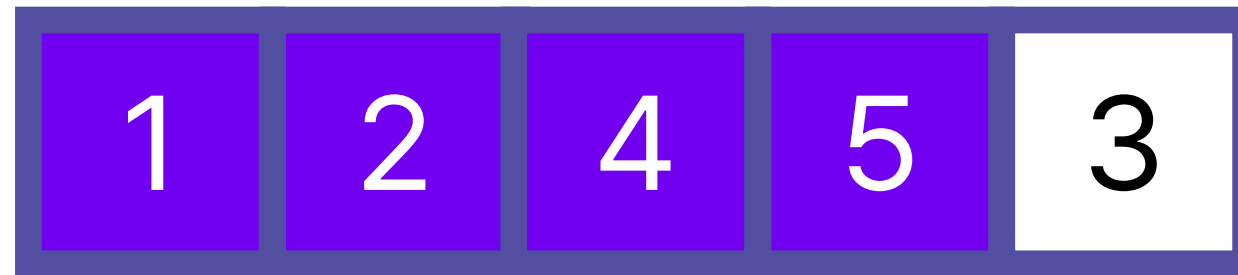


하나는 삽입할 곳을 찾는 것이고,

하나는 수를 이동시켜 실제 삽입을 진행해야 하는 것



삽입 정렬 (Insertion Sort)



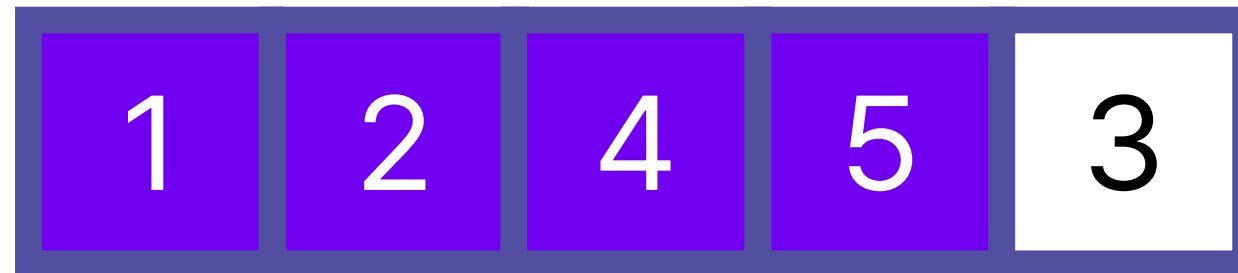
삽입할 곳을 찾는 것은 어렵지 않음

앞에서부터 수를 탐색

다만, 실제 삽입하는 것은?



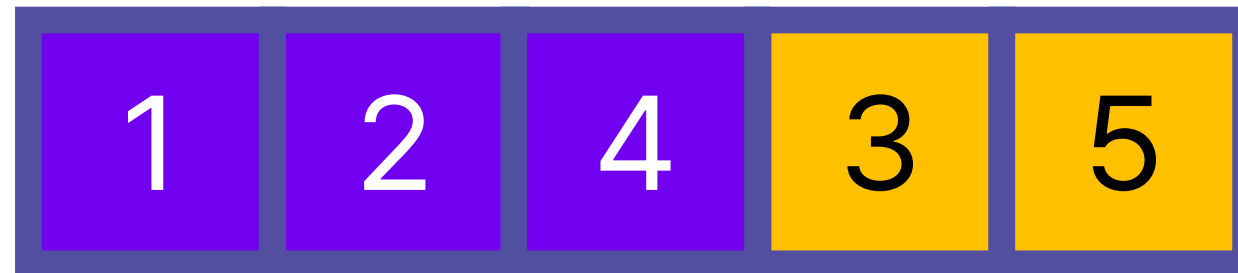
삽입 정렬 (Insertion Sort)



잘 생각해보면, 다른 수의 순서는 그대로 유지한 채,
3만 이동해야 하는 것을 알 수 있음



삽입 정렬 (Insertion Sort)



이 경우엔, 맨 뒤에서 시작해서 계속 두 수를 바꿔주면 해결됨

실제로 5와 3을 바꿔주면 배열은 다음과 같이 됨



삽입 정렬 (Insertion Sort)



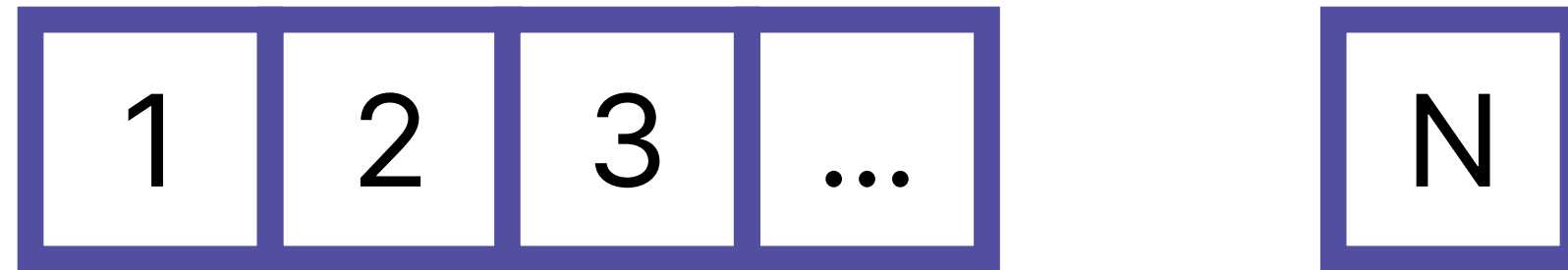
이어서, 4와 3도 바꿔보면?

3이 원래 가야 하는 위치까지 계속 변경하게 되면,

다음과 같이 모든 수가 제대로 된 순서로 놓이는 걸 볼 수 있음



삽입 정렬의 시간 복잡도

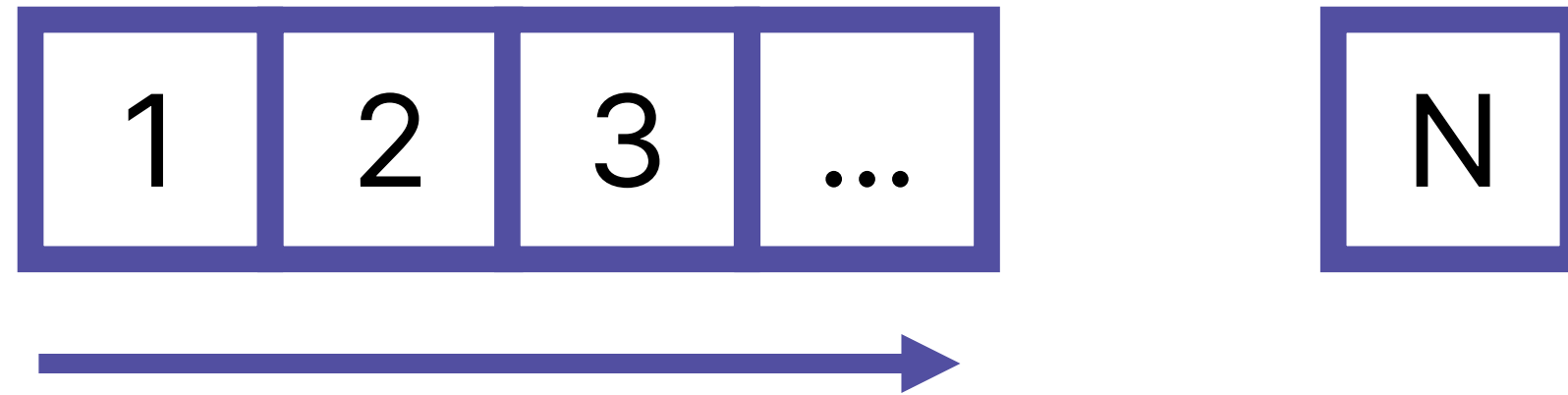


$N - 1$ 길이의 배열이 이미 정렬되어 있고,
마지막으로 N 번째 원소를 삽입하려고 한다고 가정해보면



삽입 정렬의 시간 복잡도

어떤 위치에 값이 들어가야 하는지 확인하는 연산의 시간복잡도는?



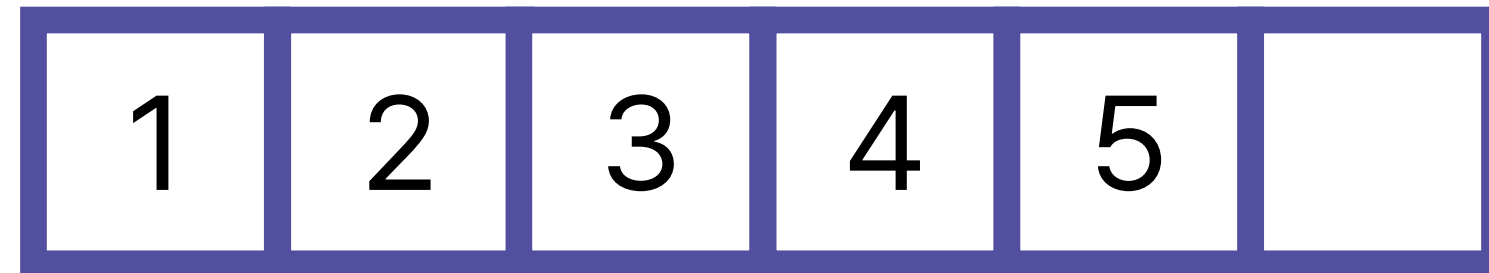
최대 $N - 1$ 번 데이터를 탐색해야 하니,

시간복잡도는 $O(N)$



삽입 정렬의 시간 복잡도

데이터를 삽입하는 연산의 시간복잡도는?

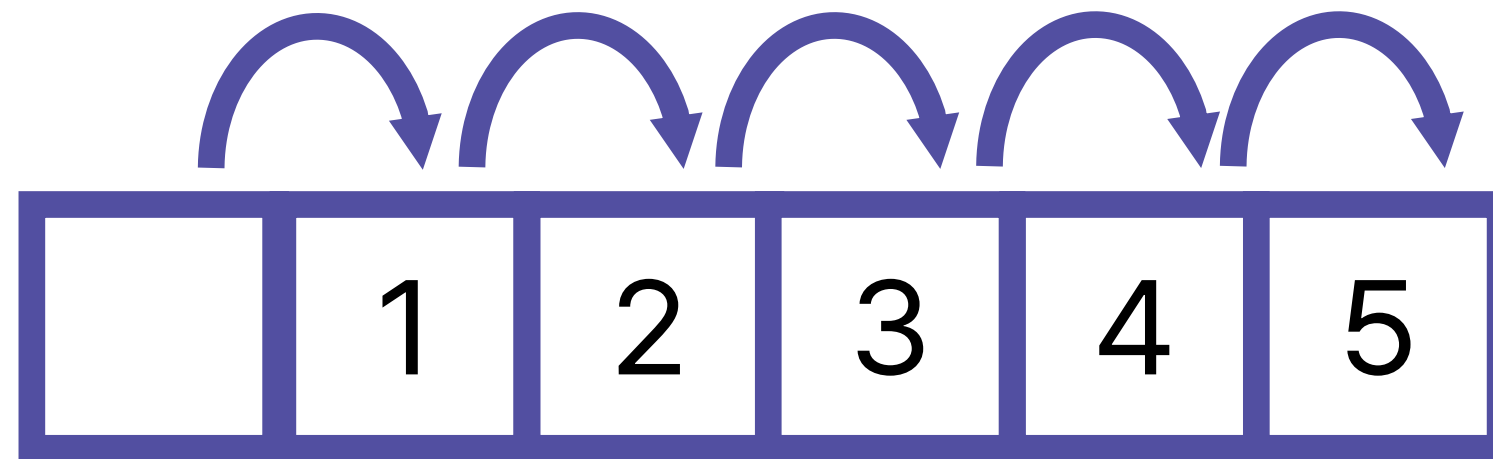


다음과 같이 배열이 있고, 맨 앞에 데이터를 삽입한다고 가정



삽입 정렬의 시간 복잡도

데이터를 삽입하는 연산의 시간복잡도는?



맨 앞에 삽입하기 위해선,

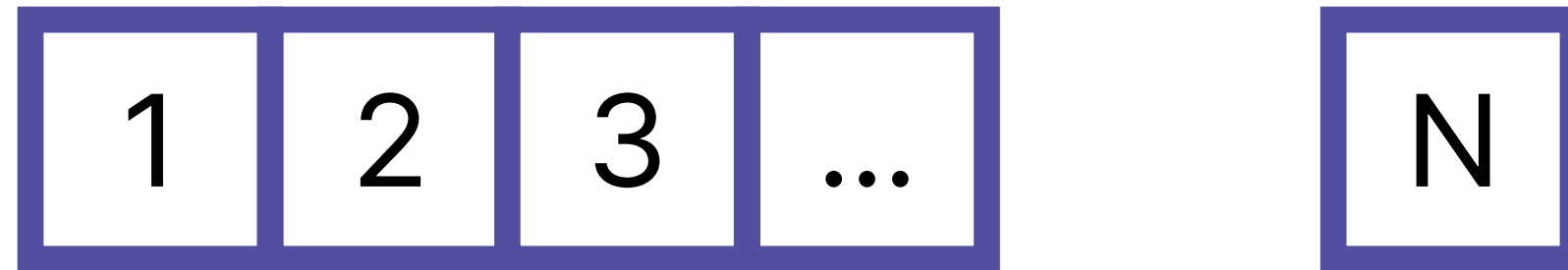
다른 데이터들을 모두 한 칸씩 이동해야 함

즉, $N - 1$ 개의 데이터를 움직여야 함

따라서, 값을 삽입하는 것 또한 $O(N)$ 의 시간복잡도



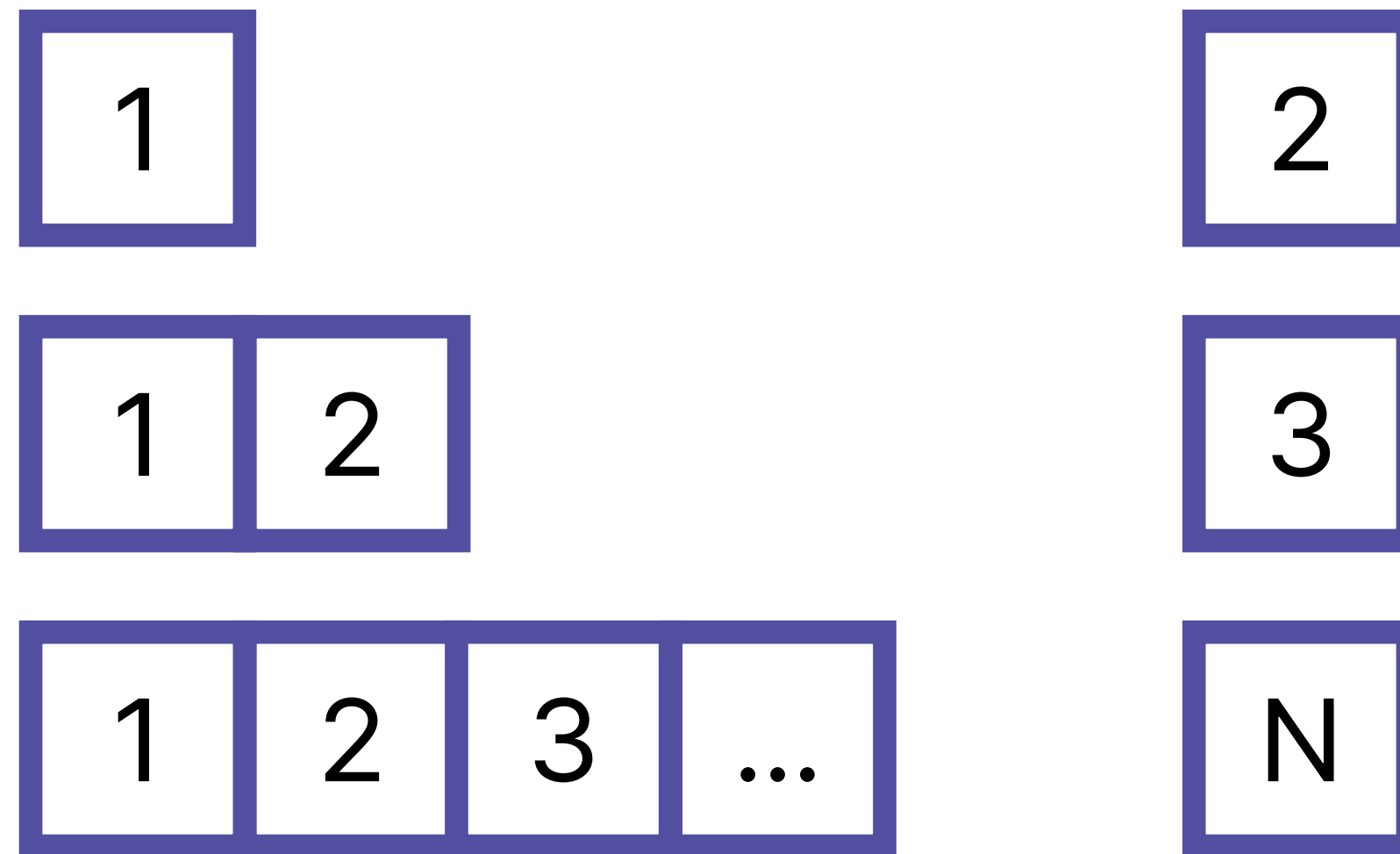
삽입 정렬의 시간 복잡도



결과적으로, 정렬된 $N - 1$ 길이의 배열에서
N번째 원소를 삽입하는 시간복잡도는 $O(N)$



삽입 정렬의 시간 복잡도

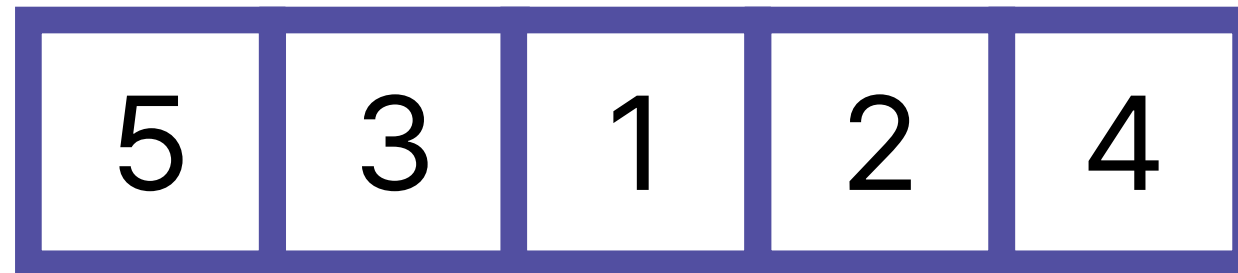


삽입 정렬의 과정을 자세히 살펴보면,
배열의 길이가 점점 늘어나는 것을 볼 수 있음
삽입 정렬은 $O(N^2)$ 의 시간 복잡도

버블 정렬 (Bubble Sort)



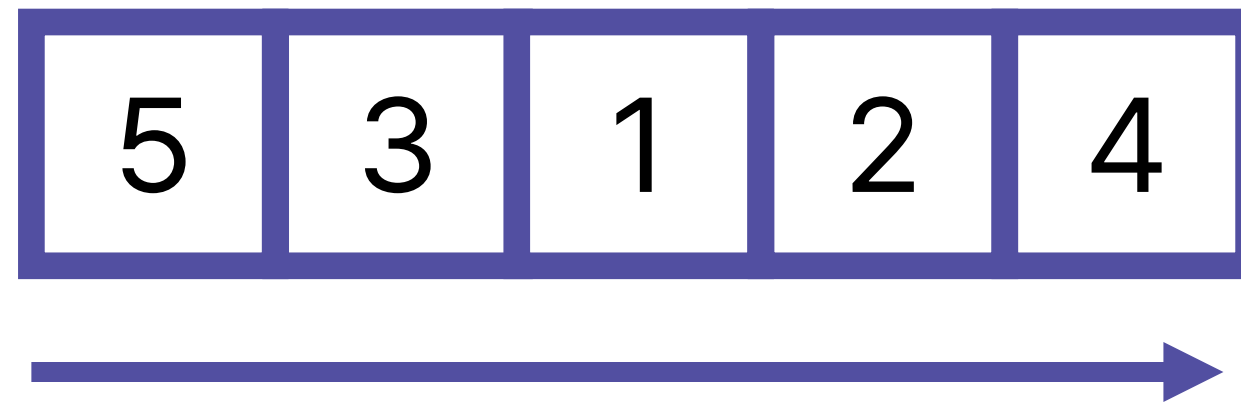
버블 정렬 (Bubble Sort)



다음과 같은 배열이 있다고 해보자



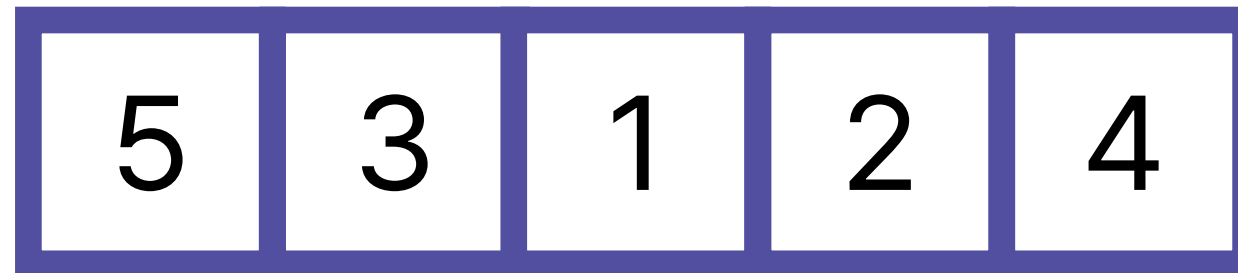
버블 정렬 (Bubble Sort)



우리는 첫 번째 원소와 두 번째 원소를 비교하고,
두 번째 원소와 세 번째 원소를 비교하고,
... 을 반복해서 마지막까지 두 수를 비교해야 함



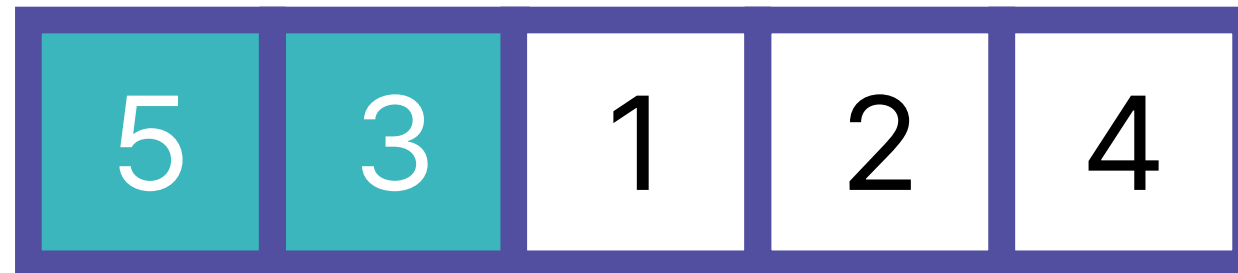
버블 정렬 (Bubble Sort)



두 수를 비교하면서, 더 큰 숫자를 뒤로 보내야 함



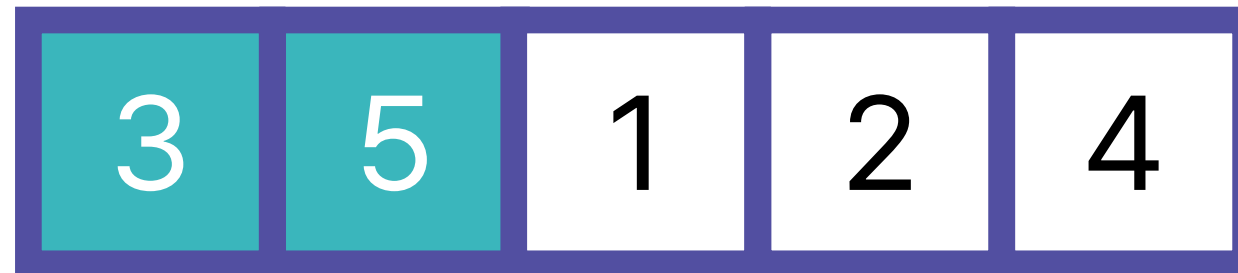
버블 정렬 (Bubble Sort)



먼저, 5와 3을 비교해보면?



버블 정렬 (Bubble Sort)



5가 3보다 더 크므로, 5가 뒤로 가야 함



버블 정렬 (Bubble Sort)



이어서, 5와 1을 비교해보면?



버블 정렬 (Bubble Sort)



이번에도 5가 1보다 더 크므로, 두 수를 교환해야 함



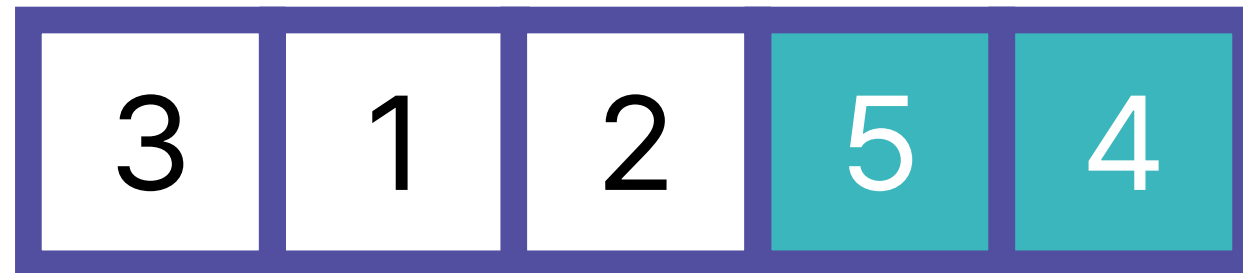
버블 정렬 (Bubble Sort)



이런 식으로 끝까지 값을 비교하면서 교체를 진행



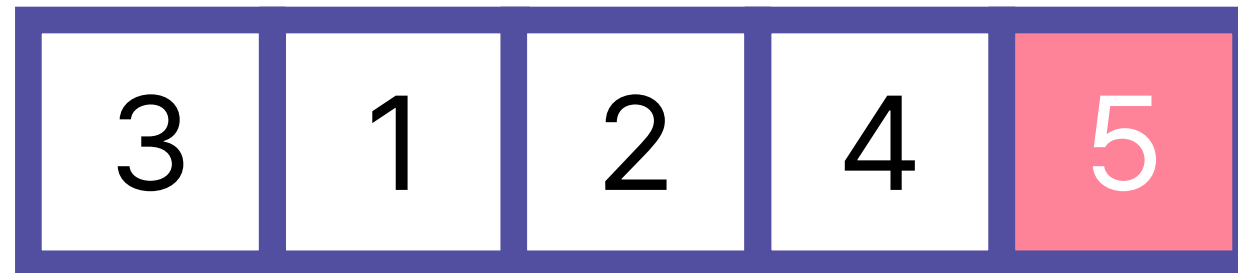
버블 정렬 (Bubble Sort)



이런 식으로 끝까지 값을 비교하면서 교체를 진행



버블 정렬 (Bubble Sort)



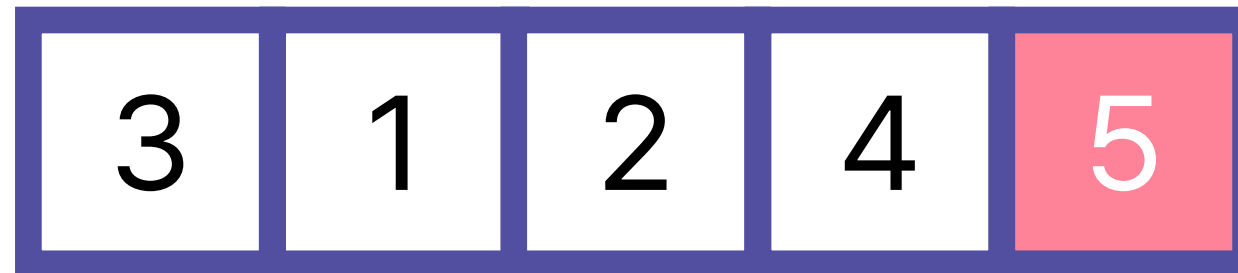
이렇게 이동을 하면, 맨 끝에 가장 큰 숫자가 놓이게 됨

이런 방식으로 진행하는 정렬 알고리즘을

버블 정렬이라고 부름



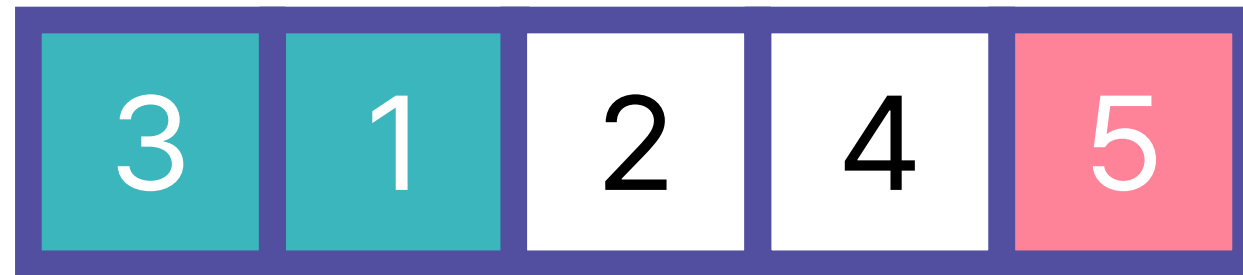
버블 정렬 (Bubble Sort)



그렇다면, 이어서 정렬을 진행해보자



버블 정렬 (Bubble Sort)

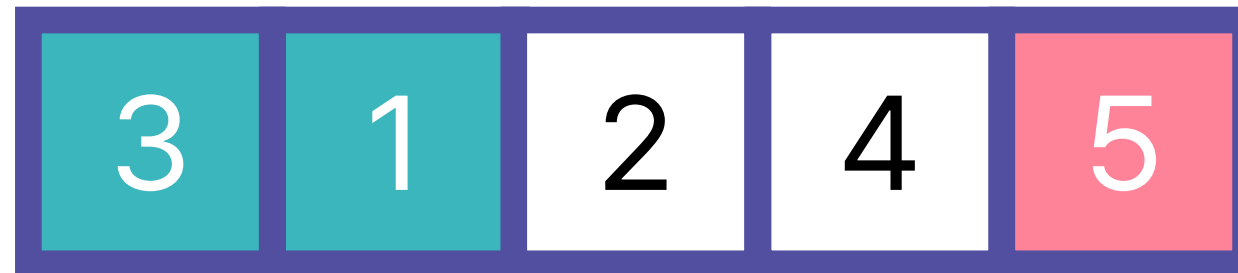


3과 1,

둘 중에 어떤 숫자가 더 큰가?



버블 정렬 (Bubble Sort)



3이 더 크므로, 자연스럽게 3이 뒤로 가야 함



버블 정렬 (Bubble Sort)



이번에는 3과 2



버블 정렬 (Bubble Sort)



마찬가지로 3이 더 크므로, 3을 뒤로 보냄



버블 정렬 (Bubble Sort)



마지막으로 3과 4가 있다면?



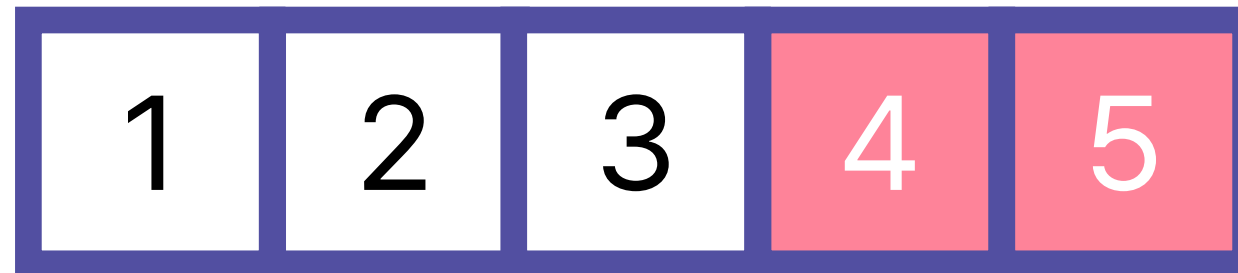
버블 정렬 (Bubble Sort)



4가 더 크기 때문에 교환은 발생하지 않음



버블 정렬 (Bubble Sort)

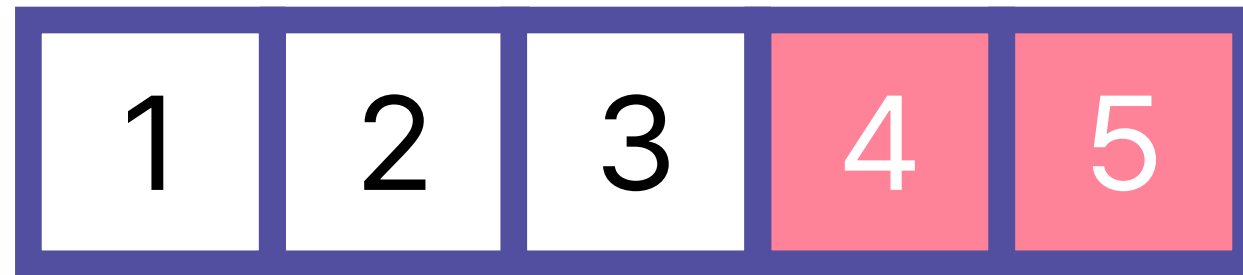


이어서 정렬을 진행해야 하는데...

현재 배열 상태는?



버블 정렬 (Bubble Sort)

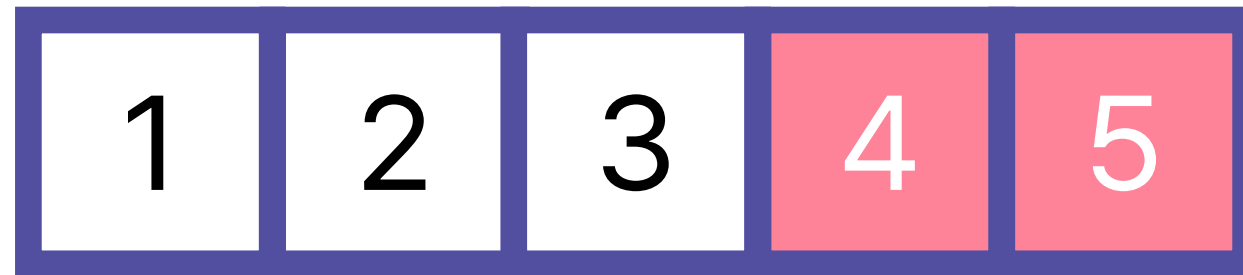


이미 정렬된 상태!

정렬 알고리즘을 끝까지 돌리지 않고 종료해야 함



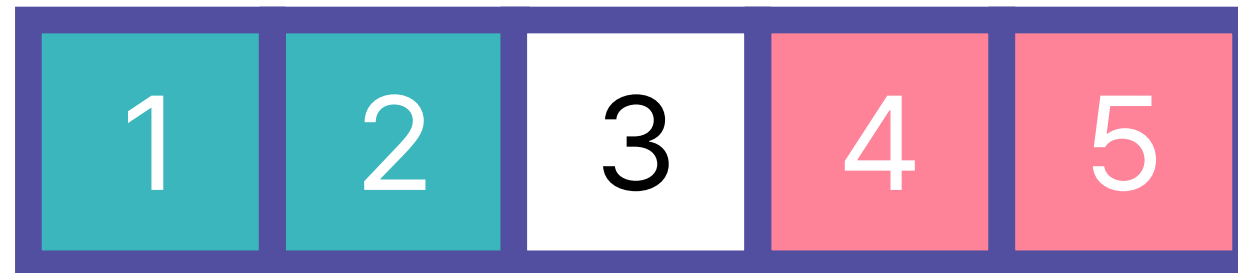
버블 정렬 (Bubble Sort)



일단 이 상태로 정렬을 시도해보면?



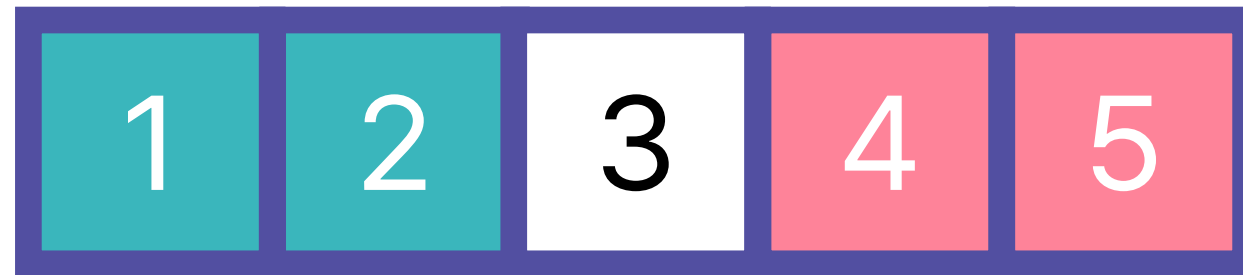
버블 정렬 (Bubble Sort)



먼저 1과 2를 비교해야 함
두 수 중 어떤 수가 더 큰가?



버블 정렬 (Bubble Sort)



2가 더 크므로, 교체는 발생하지 않음



버블 정렬 (Bubble Sort)



이번에는 2와 3을 비교해보면?



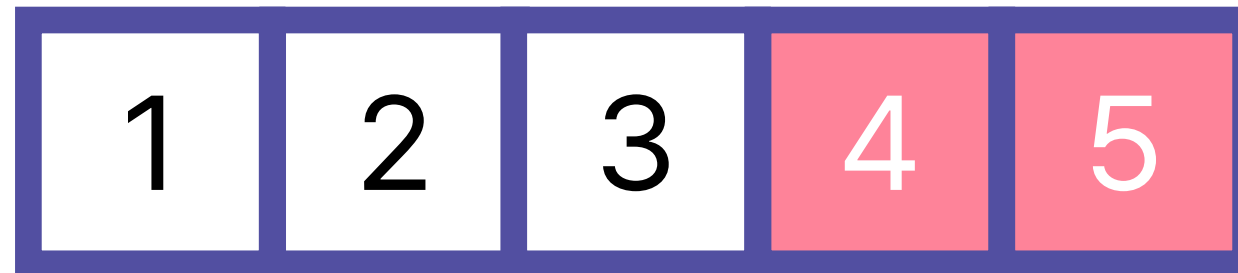
버블 정렬 (Bubble Sort)



마찬가지로 3이 더 크므로 교체는 발생하지 않음



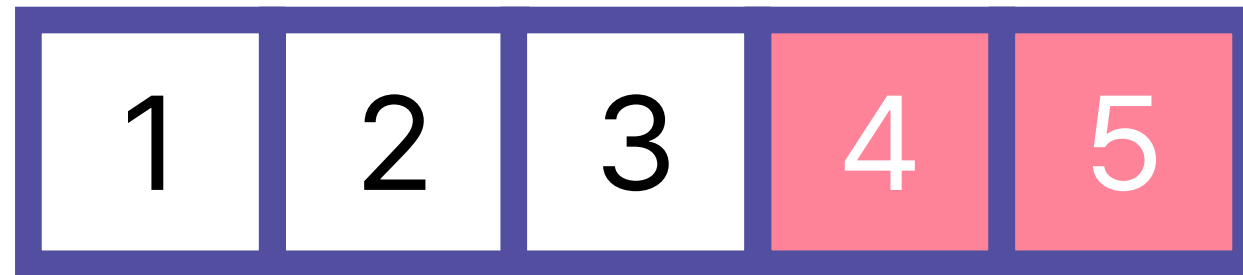
버블 정렬 (Bubble Sort)



배열이 정렬된 상태면 교체가 발생하지 않는다!



버블 정렬 (Bubble Sort)

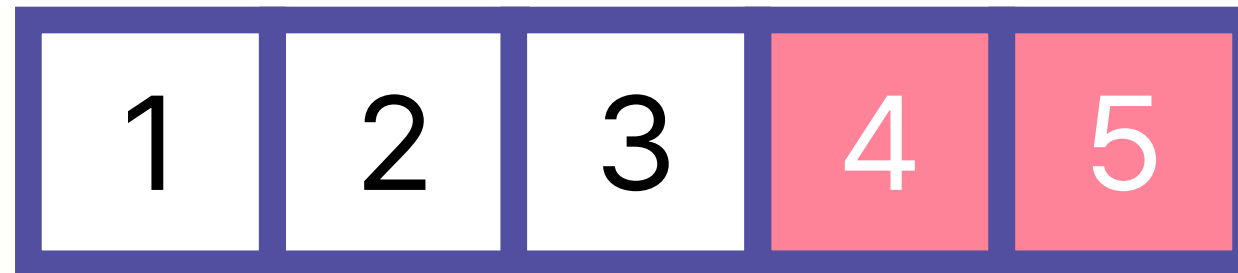


즉, 우리가 반복문을 한 번 수행했을 때,

값 교환이 발생하지 않았다면 배열이 정렬되었다고 볼 수 있음



버블 정렬 (Bubble Sort)



즉, 반복문을 한 번 돌고나서 **값 교환이 발생했는지 체크**하고,

만약 발생하지 않았다면 중간에 함수를 종료



버블 정렬 (Bubble Sort)

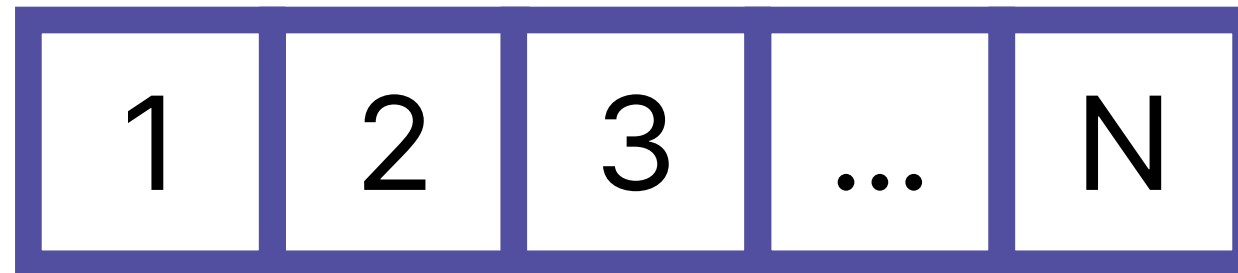
```
for i in range(N):  
    flag = True  
    for j in range(1, N - i):  
        # 값을 비교하고, 교환  
        if arr[j - 1] > arr[j]:  
            arr[j - 1], arr[j] = arr[j], arr[j - 1]  
            flag = False  
  
    if flag:  
        break
```

그래서, 각 반복문을 돌기 전 교환을 했는지

체크하는 변수를 만들어주면 좋음



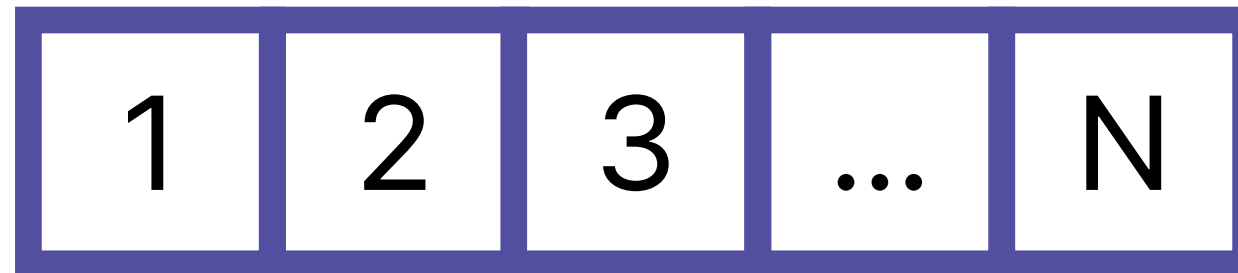
버블 정렬의 시간 복잡도



다음과 같은 배열이 있다고 가정해보자



버블 정렬의 시간 복잡도

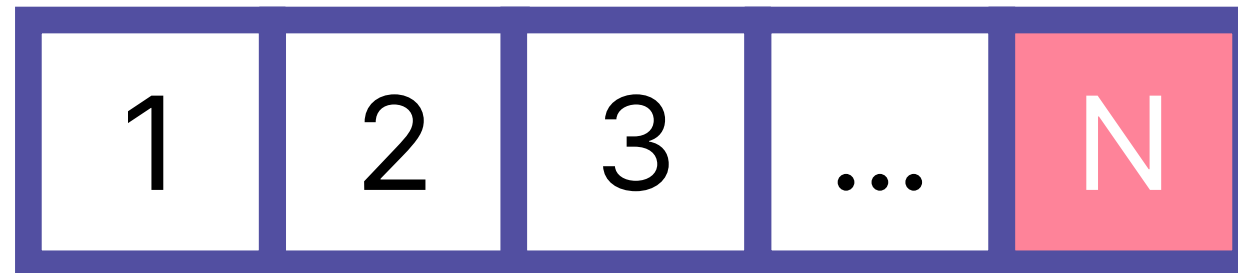


정렬이 안 된 배열의 길이가 N이라고 가정하면,

몇 번의 값 비교를 진행해야 할까? $N - 1$ 번



버블 정렬의 시간 복잡도

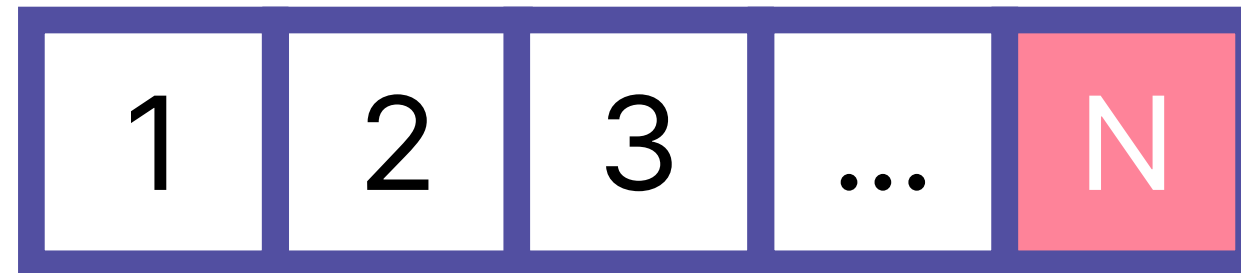


반복문을 한 번 돌면, 남은 배열의 길이는 $N - 1$ 이 됨

그렇다면 이번엔 몇 번의 비교가 필요할까?



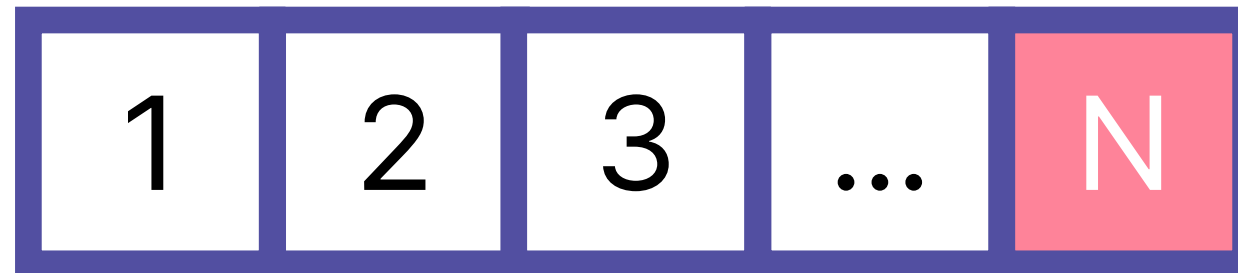
버블 정렬의 시간 복잡도



이번엔 $N - 2$ 번



버블 정렬의 시간 복잡도



선택 정렬과 마찬가지로, 한 번 반복문을 돌 때마다
정렬을 해야 하는 데이터가 하나씩 줄어드는 것을 볼 수 있음



버블 정렬의 시간 복잡도

즉, 정렬이 완료되기 위해 필요한 비교의 횟수는

$$(N - 1) + (N - 2) + \dots 2 + 1 \text{ 번}$$



버블 정렬의 시간 복잡도

선택 정렬에서 합을 구했던 것을 생각하면,
대략 $O(N^2)$ 의 비교를 진행해야 한다고 볼 수 있음



버블 정렬의 시간 복잡도

필요한 비교의 횟수 $O(N^2)$

그렇다면, 비교 연산의 시간복잡도는 어떻게 될까?



버블 정렬의 시간 복잡도

필요한 비교의 횟수 $O(N^2)$

단순히 값을 비교하고, 대소관계에 따라
값을 교환하는 것이 연산의 전부였음



버블 정렬의 시간 복잡도

필요한 비교의 횟수 $O(N^2)$

비교 연산 $O(1)$

두 연산은 모두 시간복잡도는 $O(1)$ 이므로,
비교 연산의 시간복잡도도 $O(1)$ 라고 볼 수 있음



버블 정렬 (Bubble Sort)

필요한 비교의 횟수 $O(N^2)$

비교 연산 $O(1)$

전체 시간복잡도 $O(N^2)$

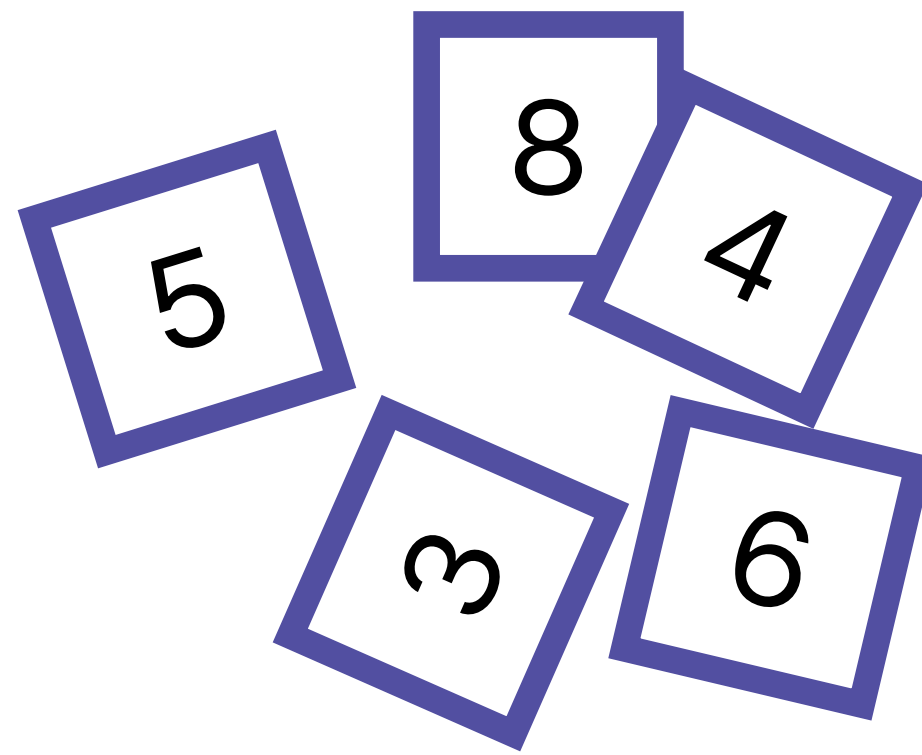
즉, $O(1)$ 연산을 $O(N^2)$ 번 수행한다는 것이므로,

시간복잡도는 $O(N^2)$ 가 됨

선택 정렬 (Selection Sort)



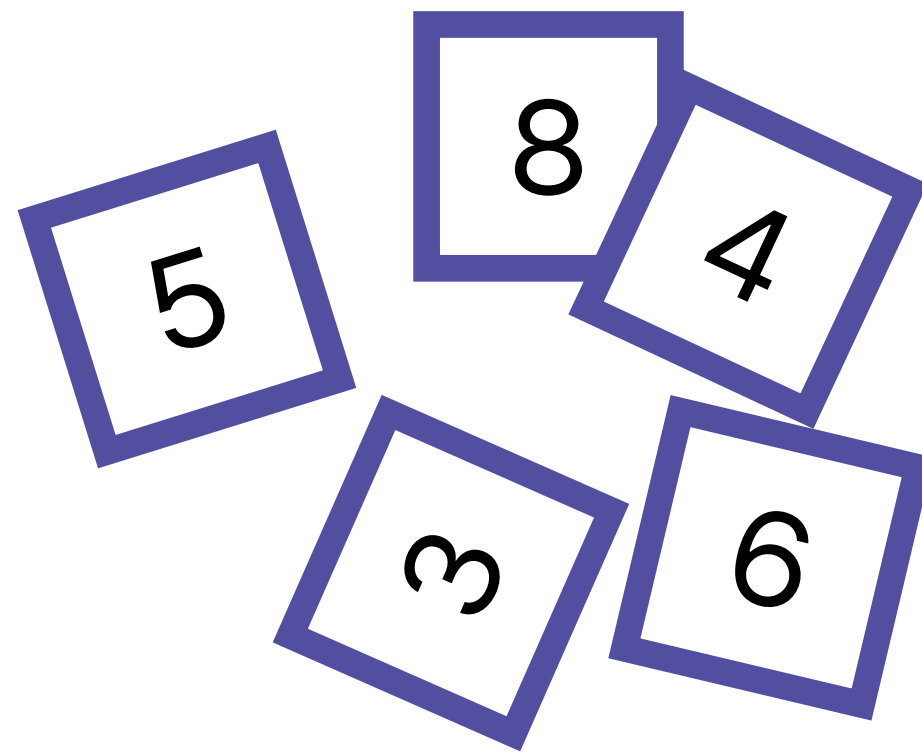
선택 정렬 (Selection Sort)



바닥에 카드들이 마구잡이로 놓여있음
순서대로 정렬하는 방법을 생각해보면?



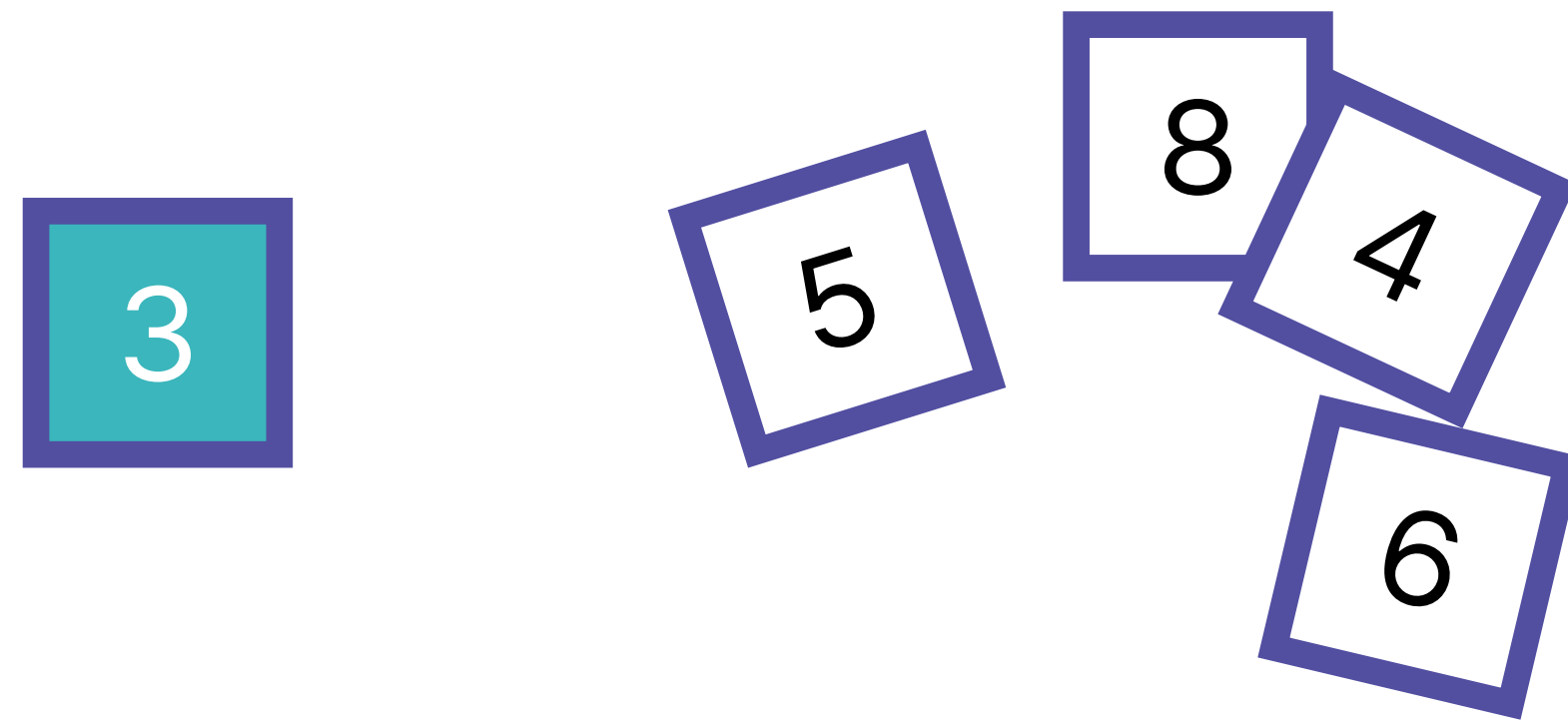
선택 정렬 (Selection Sort)



한 가지 방법은, 놓여있는 카드 중
가장 작은 카드를 고르는 것을 반복하는 것



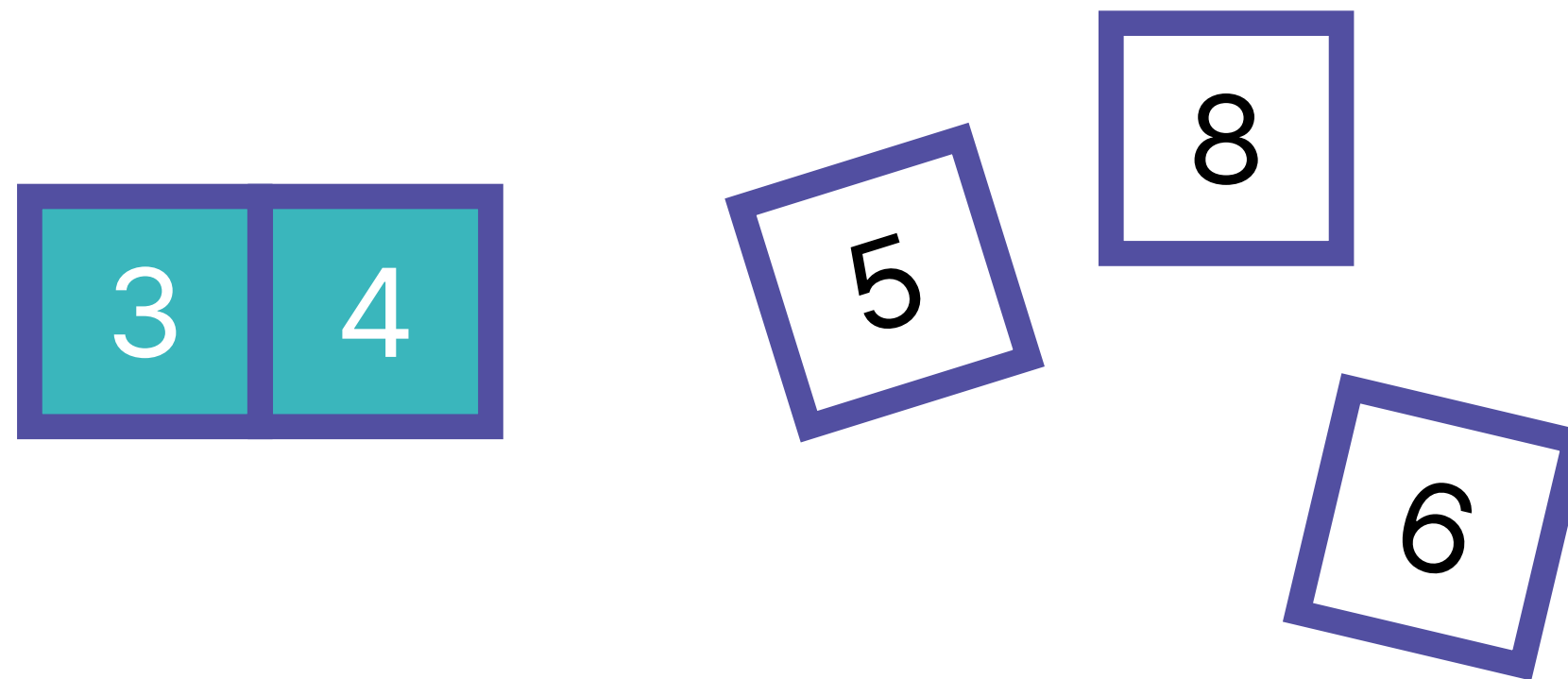
선택 정렬 (Selection Sort)



위 카드를 예시로 들자면,
가장 먼저 제일 작은 숫자인 3을 고를 수 있음



선택 정렬 (Selection Sort)



이어서, 남은 카드 중 가장 작은 숫자는 4이므로,
4를 빼도록 함



선택 정렬 (Selection Sort)



계속해서, 5 – 6 – 8 순서대로 카드를 고르게 되면,

다음과 같이 카드가 정렬된 상태로 배치됨



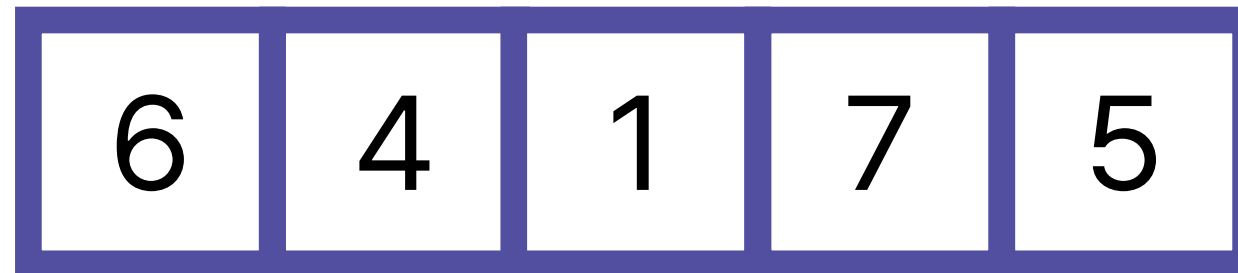
선택 정렬 (Selection Sort)



이런 식으로 가장 작은 값을 반복적으로 **선택**하는 정렬 방식을
컴퓨터에선 **선택 정렬 알고리즘**이라고 부름



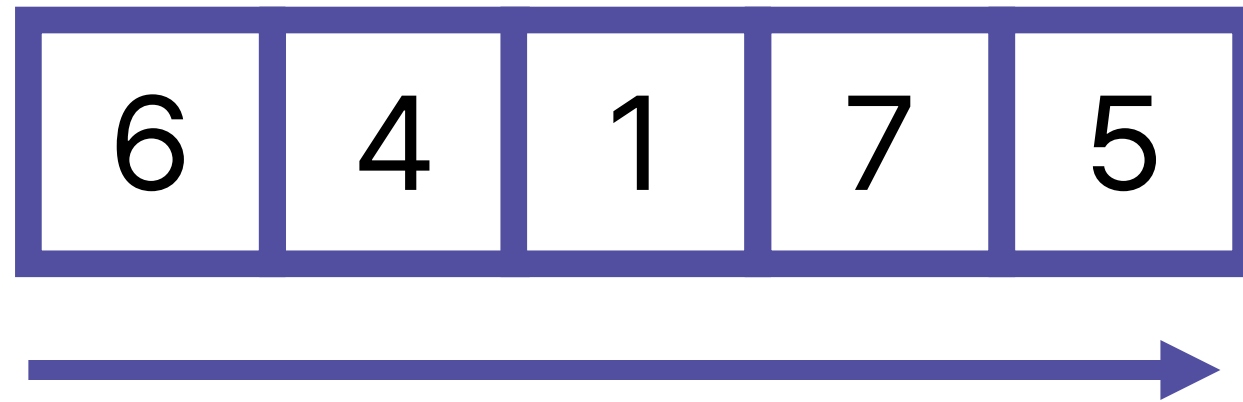
선택 정렬 (Selection Sort)



배열에서 최솟값을 찾는 방법은?



선택 정렬 (Selection Sort)



최솟값을 찾기 위해선,
반복문을 사용해 전체 배열의 값을 한 번씩 확인해야 함



선택 정렬 (Selection Sort)



최솟값을 찾았다면,
이 값을 맨 앞으로 보내줘야 함



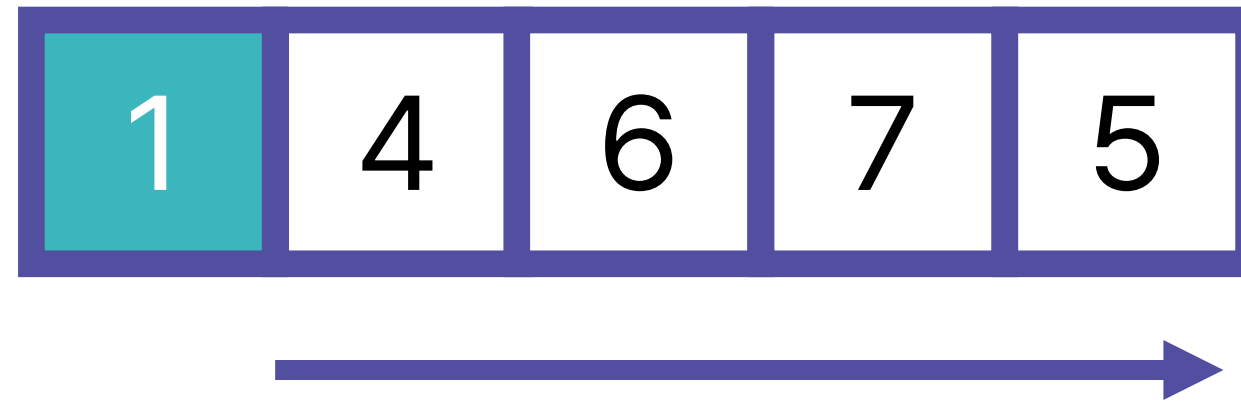
선택 정렬 (Selection Sort)



즉, 6과 1을 서로 바꿔야 함



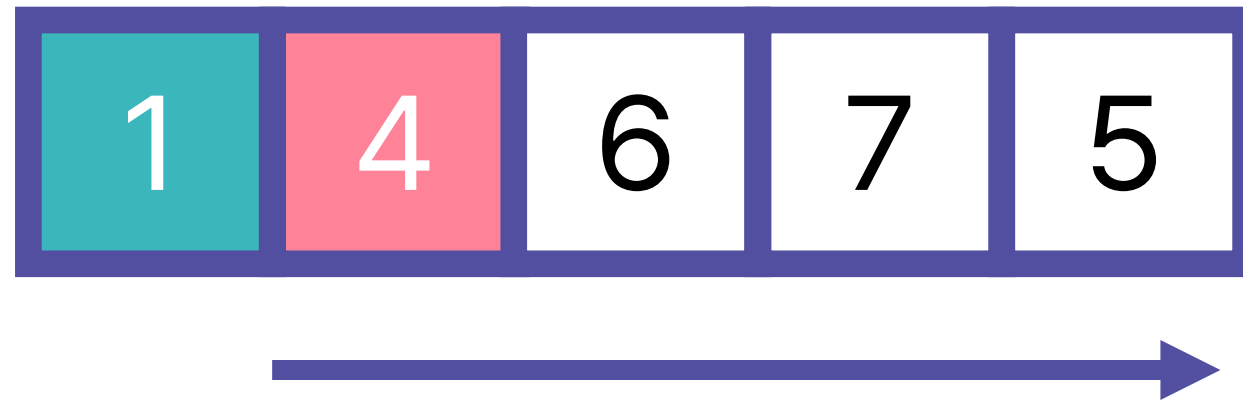
선택 정렬 (Selection Sort)



이어서, 남은 값 중에서 가장 작은 값을 찾아야 함



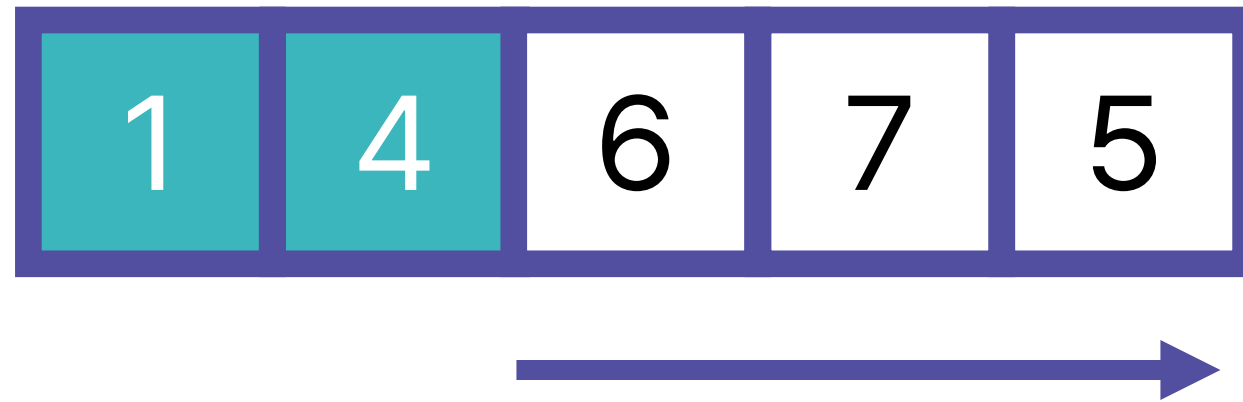
선택 정렬 (Selection Sort)



가장 작은 값은 4이므로,
특별하게 값을 교체하지 않고 넘어가도 됨



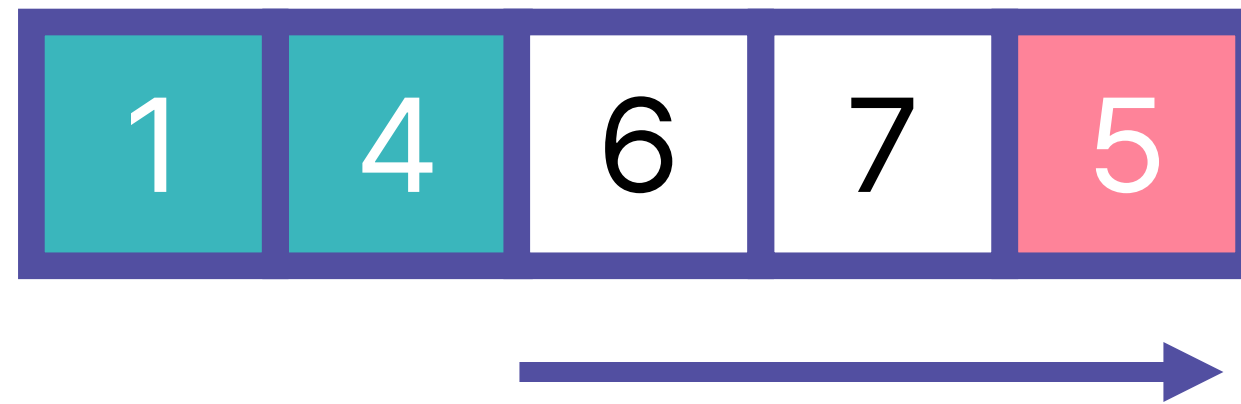
선택 정렬 (Selection Sort)



이제 남은 값 중에서 가장 작은 값은 무엇일까?



선택 정렬 (Selection Sort)



이제 남은 값 중에서 가장 작은 값은 무엇일까?

5가 될 것!



선택 정렬 (Selection Sort)



앞에서 했던 것처럼, 6과 5를 교체



선택 정렬 (Selection Sort)



이제 두 개의 값만 남음

둘 중 더 작은 값은?



선택 정렬 (Selection Sort)



6이 더 작으므로, 7과 6을 교체



선택 정렬 (Selection Sort)



남은 값은 한 개인데,
그것 자체로 가장 작은 값



선택 정렬 (Selection Sort)



따라서, 마지막 값은 따로 움직이지 않아도 괜찮음

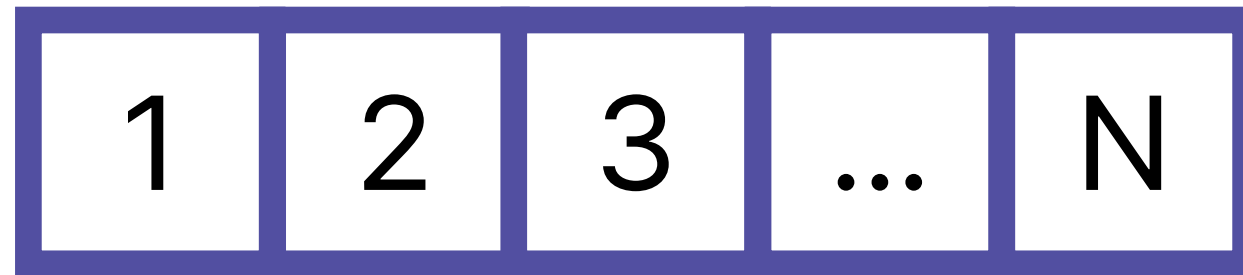


선택 정렬은 크게 두 부분으로 나뉨

1. 남아있는 데이터 중 가장 작은 값을 찾는 작업
2. 두 값을 교체하는 작업



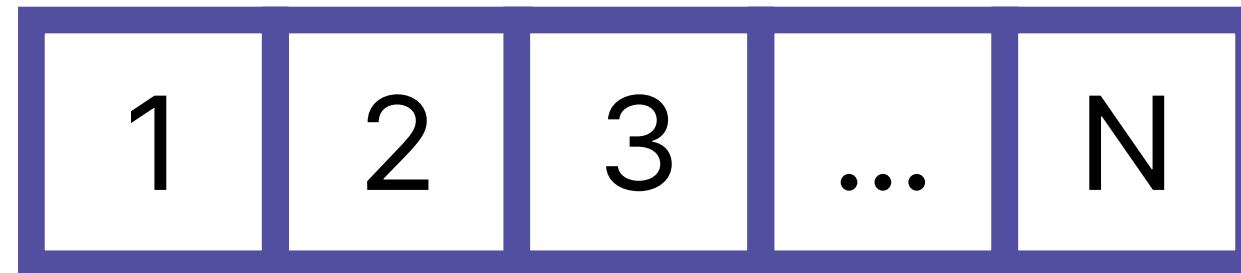
선택 정렬의 시간 복잡도



가장 작은 값을 찾는 작업을 중심으로 확인



선택 정렬의 시간 복잡도

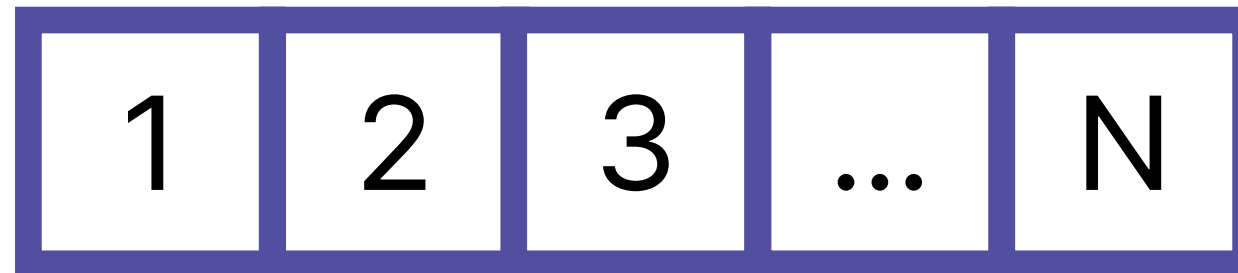


처음에 N개의 데이터가 있다면,

가장 작은 값을 찾기 위해선 몇 개의 데이터를 확인해야 할까?



선택 정렬의 시간 복잡도

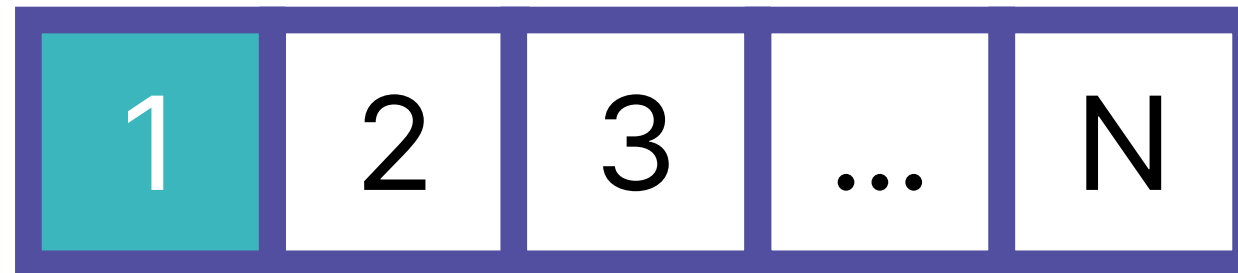


최솟값을 찾기 위해선 모든 값을 확인해야 하므로,

자연스럽게 **N개**의 데이터를 확인해야 함



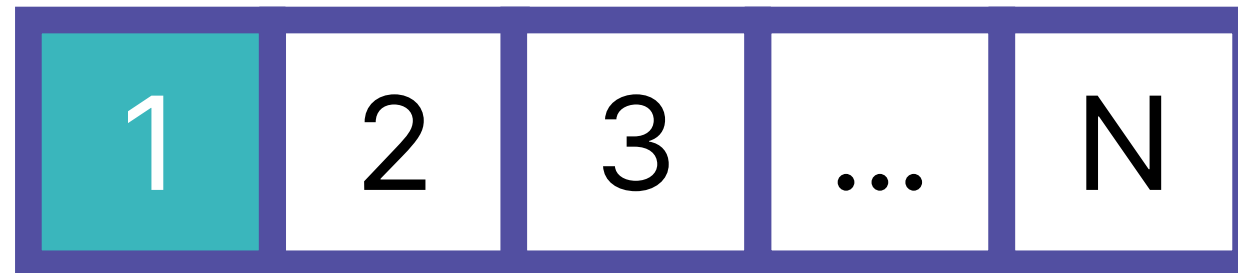
선택 정렬의 시간 복잡도



이제 데이터의 수가 $N - 1$ 개가 됨



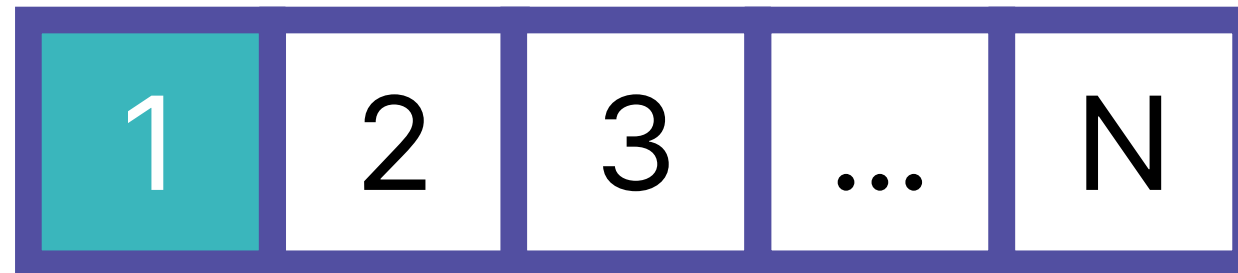
선택 정렬의 시간 복잡도



이번엔 가장 작은 값을 찾기 위해
몇 개의 데이터를 확인해야 할까?



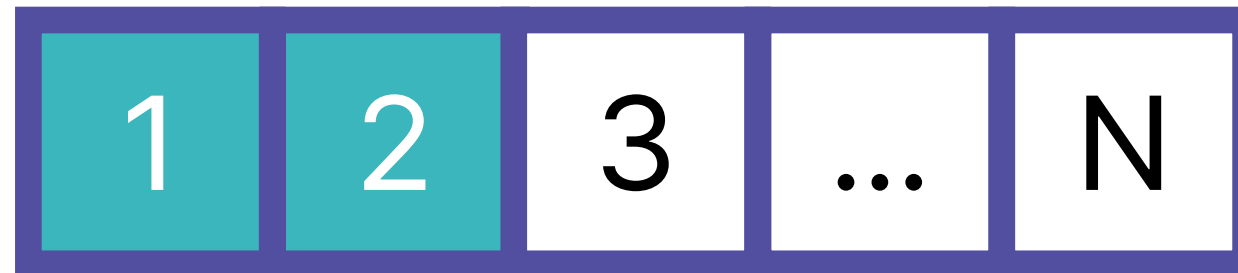
선택 정렬의 시간 복잡도



데이터의 수가 $N - 1$ 개가 되었으니,
자연스럽게 $N - 1$ 개의 데이터를 확인해야 함



선택 정렬의 시간 복잡도



이렇게 탐색하는 데이터의 수가
하나씩 줄어들고 있는 것을 볼 수 있음



선택 정렬의 시간 복잡도

즉, 정렬이 완료되기 위해 탐색하는 값의 수는

$N + (N - 1) + (N - 2) + \dots 2 + 1$ 번이 될 것



선택 정렬의 시간 복잡도

$$\sum_{i=1}^N i = 1 + 2 + \dots + (N-1) + N = \frac{N(N-1)}{2}$$

합을 구하면, $(N * (N - 1)) / 2$



선택 정렬의 시간 복잡도

$$\sum_{i=1}^N i = 1 + 2 + \dots + (N-1) + N = \frac{N(N-1)}{2}$$

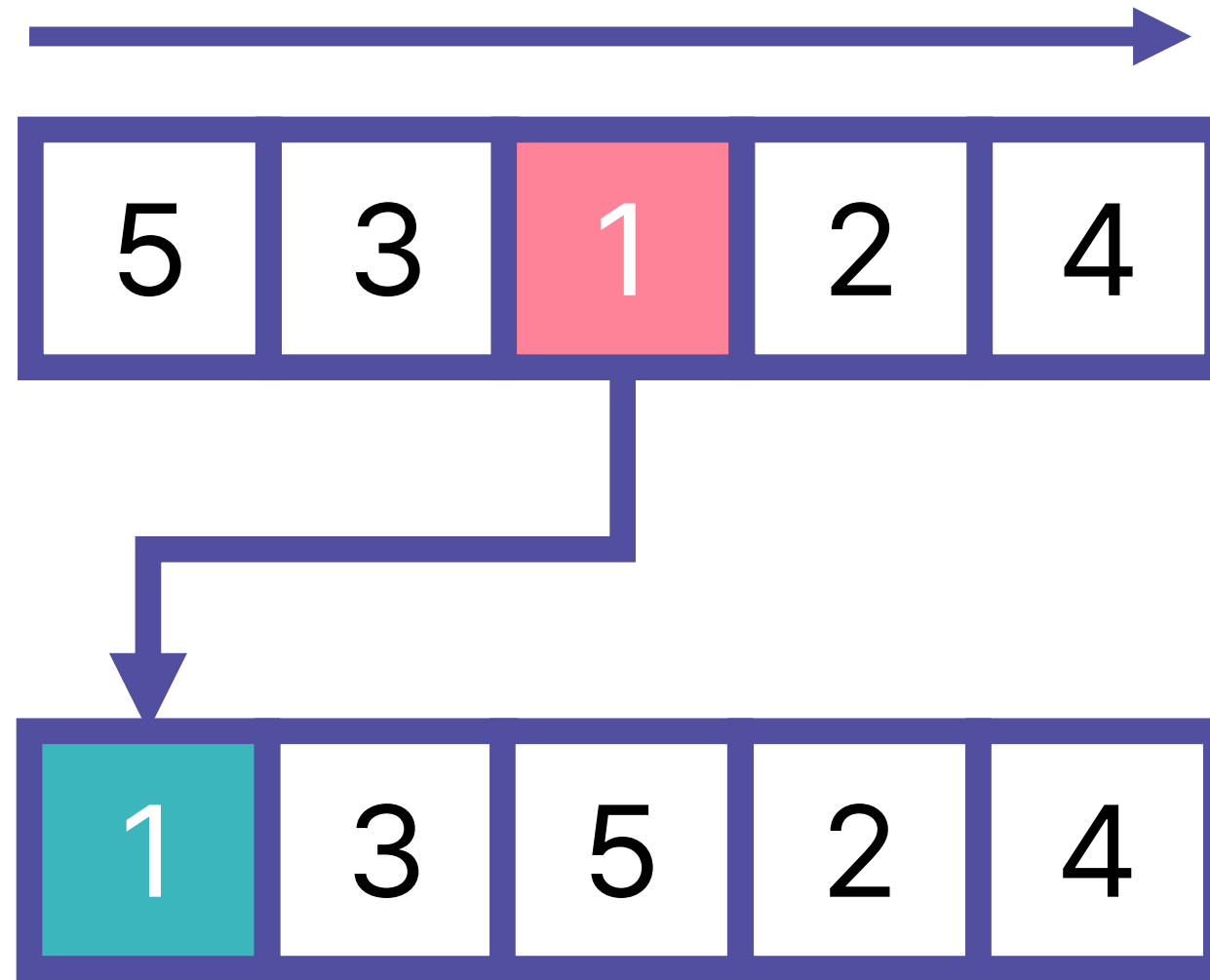
이 값을 점근 표기법을 사용해 표현하면, $O(N^2)$



선택 정렬과 버블 정렬의 시간복잡도는 모두 $O(N^2)$ 인데,
둘 사이의 어떤 차이가 있을까?



선택 정렬과 버블 정렬

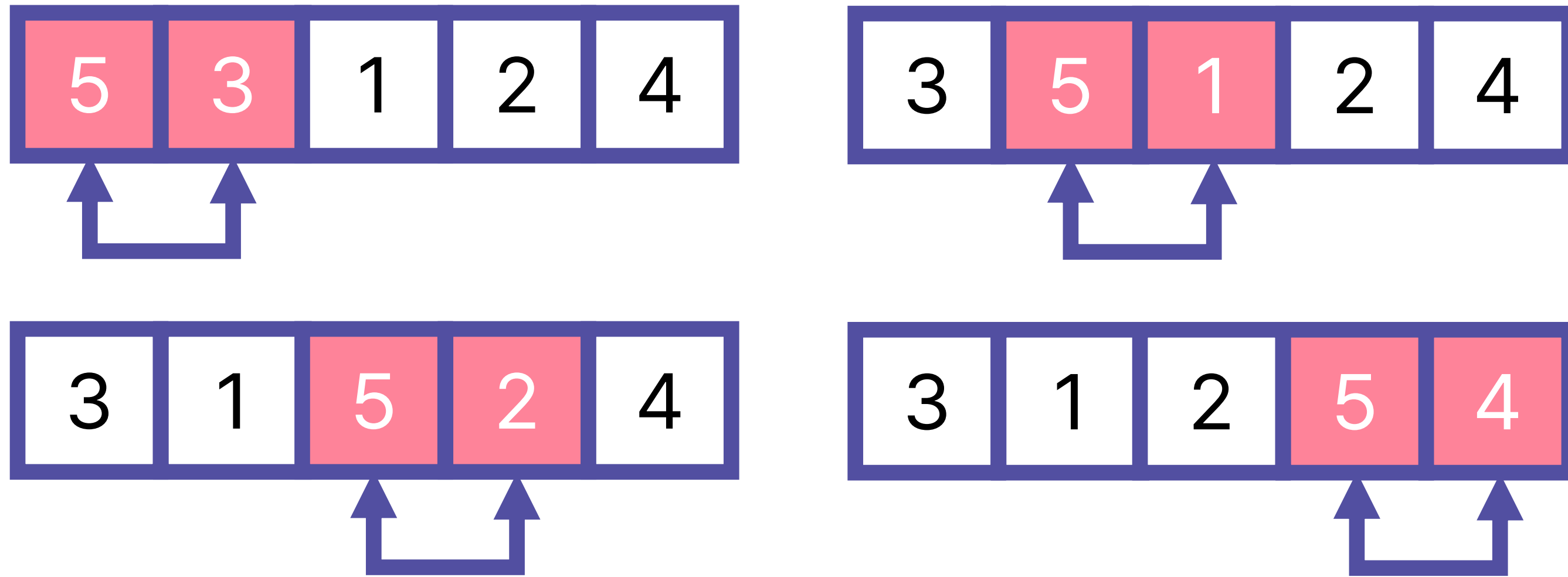


선택 정렬은 배열 전체를 돌면서

가장 작은 값을 찾아 배열의 맨 앞으로 보내는 것을 반복했음



선택 정렬과 버블 정렬

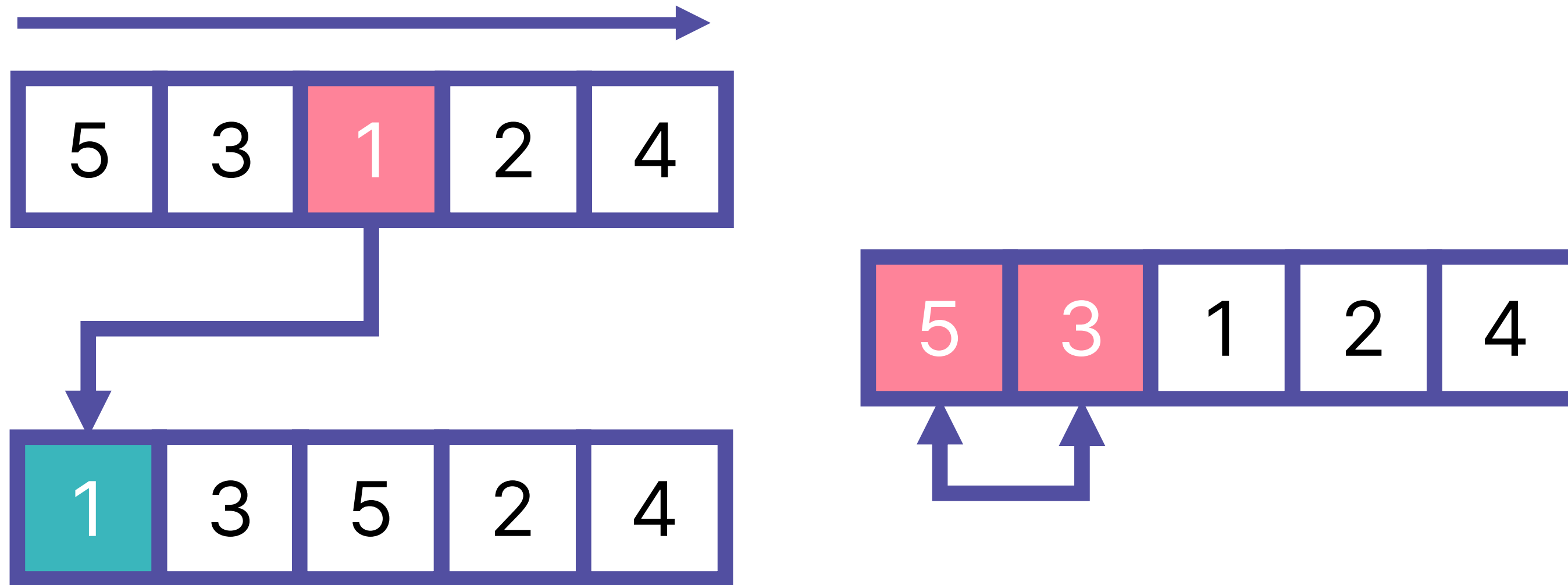


반면, 버블 정렬은 값 하나를 찾기 위해 배열 전체를 순회하지 않고,

두 값을 비교하고 교환하는 것을 반복해 최댓값을 끝으로 보냄



선택 정렬과 버블 정렬



두 알고리즘 모두 최댓값/최솟값을

한 방향으로 채우는 방식이지만, 실제 동작 과정은 매우 다름